Hubbard-Holstein Mathematical Implementation (Current Linear Substitution-First Form)

Jake Skyler Strobel (repo-grounded revision)

March 14, 2026

1. Parameter Manifest and Reader Contract

This manuscript is a present-tense, self-contained mathematical description of the implemented Hubbard and Hubbard-Holstein (HH) stack in this repository. It keeps the same linear style as the older manuscript,

1. primitives first,
2. composite operators second,
3. explicit substitutions third,
4. fully substituted forms last,

but it removes older future-tense framing and replaces it with the currently implemented operator, variational, drive, PAOP, continuation, and handoff surfaces.

1.1 Required parameter manifest

- Model family: `Hubbard` / `Hubbard-Holstein`.
- Lattice size: `L` or `dims`.
- Fermion ordering: `blocked` or `interleaved`.
- Boundary condition: `open` or `periodic`.
- Core fermion parameters: `t` (or `J` on HH surfaces), `U`, `dv` / site potentials.
- Core phonon parameters: `omega0`, `g` / `g_ep`, `n_ph_max`, `boson_encoding`.
- Variational family when relevant:
  - `hh_hva` = HH layerwise,
  - `hh_hva_tw` = HH Pauli-termwise,
  - `hh_hva_ptw` = HH physical-termwise,
  - ADAPT / explicit-family surfaces including `hva`, `full_meta`, `paop_*`, `uccsd_paop_lf_full`, `uccsd_otimes_paop_lf_std`, `uccsd_otimes_paop_lf2_std`, `uccsd_otimes_paop_bond_disp_std`, optional logical-pair `*_seq2p`, and `full_hamiltonian`.
- Optimizer/runtime parameters when relevant:
  - VQE optimizer,
  - SPSA schedule parameters,
  - energy backend,
  - ADAPT state backend,
  - staged continuation mode,
  - drive waveform controls,
  - propagator choice.

1.2 Reader contract

This document intentionally favors explicit substitution over compressed meta-definition.

- If a primitive exists, it is written first.
- If a composite operator is formed from primitives, those primitives are substituted into the composite.
- If a later formula can be reduced by inserting an earlier primitive explicitly, the insertion is shown.

- When a runtime surface has different defaults from a core builder surface, the difference is named explicitly instead of being hidden behind a single "repo default".

1.3 Non-negotiable repository conventions

1.3.1 Internal Pauli alphabet

Internally the Pauli alphabet is always

$$\{e, x, y, z\},$$

with `e` as identity.

1.3.2 Pauli-word and qubit ordering

Pauli words and computational-basis labels are written left-to-right as

$$q_{N_q-1} \cdots q_1 q_0,$$

with qubit `q_0` the rightmost character and also the least-significant bit in basis-index arithmetic.

1.3.3 Canonical algebra sources

- Canonical `PauliTerm`: `src/quantum/qubitization_module.py`
- Canonical `PauliPolynomial`: `src/quantum/pauli_polynomial_class.py`
- Canonical JW ladder primitives:
  - `fermion_plus_operator(...)`
  - `fermion_minus_operator(...)`
- Canonical number operator surface:
  - `jw_number_operator(...)` in `src/quantum/hubbard_latex_python_pairs.py`

1.3.4 Surface-specific defaults

This manuscript does **not** pretend there is one universal default surface.

- Core builder functions in `src/quantum/hubbard_latex_python_pairs.py` often default to
  - `indexing="interleaved"`,
  - `pbc=True`.
- Hardcoded pipeline CLIs use different defaults, most notably
  - `ordering="blocked"`,
  - `boundary="periodic"`.
- `pipelines/hardcoded/adapt_pipeline.py` currently exposes HH `--boson-encoding binary` only.
- `pipelines/hardcoded/hubbard_pipeline.py` exposes a broader boson-encoding surface.

Whenever a formula depends on the surface, the surface is named locally.

1.4 Canonical code anchors

- `AGENTS.md`
- `README.md`
- `src/quantum/qubitization_module.py`
- `src/quantum/pauli_polynomial_class.py`
- `src/quantum/hubbard_latex_python_pairs.py`
- `src/quantum/hartree_fock_reference_state.py`
- `src/quantum/vqe_latex_python_pairs.py`
- `src/quantum/operator_pools/polaron_paop.py`
- `src/quantum/ed_hubbard_holstein.py`
- `src/quantum/spsa_optimizer.py`
- `src/quantum/drives_time_potential.py`
- `pipelines/hardcoded/hubbard_pipeline.py`
- `pipelines/hardcoded/adapt_pipeline.py`

- `pipelines/hardcoded/hh_continuation_stage_control.py`
- `pipelines/hardcoded/hh_continuation_scoring.py`
- `pipelines/hardcoded/handoff_state_bundle.py`
- `pipelines/exact_bench/cross_check_suite.py`
- `pipelines/exact_bench/hh_noise_hardware_validation.py`

2. Ordering, Indexing, and Register Layout

2.1 Site, spin, and mode indices

The site index is

$$i \in \{0, 1, \dots, L-1\},$$

and the spin label is stored as

$$\sigma \in \{\uparrow, \downarrow\} \equiv \{0, 1\}.$$

The fermion mode index is the JW qubit index.

2.1.1 Interleaved ordering

The interleaved map is

$$p(i, \sigma) = 2i + \sigma.$$

So

$$p(i, \uparrow) = 2i, \qquad p(i, \downarrow) = 2i + 1.$$

2.1.2 Blocked ordering

The blocked map is

$$p(i, \uparrow) = i, \qquad p(i, \downarrow) = L + i.$$

These are the two cases implemented by `mode_index(...)`.

2.2 Pauli-word placement and basis-index extraction

If a Pauli letter acts on qubit `q`, then in a printed word of length `N_q` it sits at string position

$$\mathrm{pos}(q) = N_q - 1 - q.$$

If the computational-basis index is `k`, then the occupation bit on qubit `q` is

$$b_q(k) = \left\lfloor \frac{k}{2^q} \right\rfloor \bmod 2 = ((k \gg q) \& 1).$$

So the printed bitstring and the integer basis index obey the same rightmost-`q_0` convention.

2.3 Full HH register layout

The fermion register uses

$$N_{\mathrm{ferm}} = 2L$$

qubits.

If the local phonon cutoff is `n_ph_max`, then the local Hilbert dimension is

$$d = n_{\mathrm{ph,max}} + 1.$$

The phonon qubits per site are

$$q_{\mathrm{pb}} = \begin{cases} \max\{1, \lceil \log_2 d \rceil\}, & \text{binary}, \\ d, & \text{unary}. \end{cases}$$

Therefore the total HH qubit count is

$$N_q = 2L + Lq_{\mathrm{pb}}.$$

In qubit-index order the register is

$$[\text{fermion qubits} \mid \text{site-0 phonon qubits} \mid \text{site-1 phonon qubits} \mid \cdots].$$

In printed bitstring order `q_(N_q-1)...q_0`, the high-index phonon blocks appear on the left, so the displayed HH basis label is read as

$$[\text{site-(L-1) phonons} \mid \cdots \mid \text{site-0 phonons} \mid \text{fermions}].$$

Implemented surfaces:

- `src/quantum/hubbard_latex_python_pairs.py`
  - `mode_index`
  - `boson_qubits_per_site`
  - `phonon_qubit_indices_for_site`
- `src/quantum/hartree_fock_reference_state.py`
  - `bitstring_qn1_to_q0`
  - `hubbard_holstein_reference state`

3. Fermionic Primitives and Direct Substitution

3.1 Jordan-Wigner ladder primitives

For mode `p`, the creation operator implemented by `fermion_plus_operator("JW", N_q, p)` is

$$\hat{c}_p^\dagger = \frac{1}{2}\, e_{N_q-1} \cdots e_{p+1} x_p z_{p-1} \cdots z_0 - \frac{i}{2}\, e_{N_q-1} \cdots e_{p+1} y_p z_{p-1} \cdots z_0.$$

The annihilation operator implemented by `fermion_minus_operator("JW", N_q, p)` is

$$\hat{c}_p = \frac{1}{2}\, e_{N_q-1} \cdots e_{p+1} x_p z_{p-1} \cdots z_0 + \frac{i}{2}\, e_{N_q-1} \cdots e_{p+1} y_p z_{p-1} \cdots z_0.$$

Equivalently, in operator notation,

$$\hat{c}_p^\dagger = \frac{1}{2}(X_p - iY_p) \prod_{r=0}^{p-1} Z_r, \qquad \hat{c}_p = \frac{1}{2}(X_p + iY_p) \prod_{r=0}^{p-1} Z_r,$$

but the repository's printed words always follow the explicit `q_(N_q-1)...q_0` ordering above.

3.2 Number primitive

The implemented number operator is

$$\hat{n}_p = \hat{c}_p^\dagger \hat{c}_p = \frac{I - Z_p}{2}.$$

This is exactly the formula returned by `jw_number_operator(...)`.

3.3 Site densities and doublon operator

If the site `i` uses fermion modes

$$p_{i\uparrow} = p(i, \uparrow), \qquad p_{i\downarrow} = p(i, \downarrow),$$

then

$$\hat{n}_{i\uparrow} = \frac{I - Z_{p_{i\uparrow}}}{2}, \qquad \hat{n}_{i\downarrow} = \frac{I - Z_{p_{i\downarrow}}}{2}.$$

So the full site density is

$$\hat{n}_i = \hat{n}_{i\uparrow} + \hat{n}_{i\downarrow} = \frac{I - Z_{p_{i\uparrow}}}{2} + \frac{I - Z_{p_{i\downarrow}}}{2} = I - \frac{1}{2}(Z_{p_{i\uparrow}} + Z_{p_{i\downarrow}}).$$

The onsite doublon operator is

$$\hat{d}_i = \hat{n}_{i\uparrow}\hat{n}_{i\downarrow} = \frac{I - Z_{p_{i\uparrow}}}{2} \cdot \frac{I - Z_{p_{i\downarrow}}}{2} = \frac{1}{4}\Big(I - Z_{p_{i\uparrow}} - Z_{p_{i\downarrow}} + Z_{p_{i\uparrow}} Z_{p_{i\downarrow}}\Big).$$

3.4 Worked ordering substitutions

For `L=3`, the two repository orderings become

3.4.1 Interleaved

$$\begin{aligned} p(0,\uparrow) &= 0, \quad p(0,\downarrow) = 1, \\ p(1,\uparrow) &= 2, \quad p(1,\downarrow) = 3, \\ p(2,\uparrow) &= 4, \quad p(2,\downarrow) = 5. \end{aligned}$$

3.4.2 Blocked

$$\begin{aligned} p(0,\uparrow) &= 0, \quad p(0,\downarrow) = 3, \\ p(1,\uparrow) &= 1, \quad p(1,\downarrow) = 4, \\ p(2,\uparrow) &= 2, \quad p(2,\downarrow) = 5. \end{aligned}$$

Implemented surfaces:

- `src/quantum/pauli_polynomial_class.py`
- `src/quantum/hubbard_latex_python_pairs.py`
- `src/quantum/qubitization_module.py`

4. Boson Primitives and Encodings

4.1 Local boson Hilbert space

At site `i`, the truncated phonon space has basis

$$\{|0\rangle_i, |1\rangle_i, \dots, |n_{\mathrm{ph,max}}\rangle_i\}.$$

The local annihilation operator is

$$\hat{b}_i = \sum_{n=1}^{n_{\mathrm{ph,max}}} \sqrt{n}\, |n-1\rangle_i \langle n|,$$

the creation operator is

$$\hat{b}_i^\dagger = \sum_{n=0}^{n_{\mathrm{ph,max}}-1} \sqrt{n+1}\, |n+1\rangle_i \langle n|,$$

the number operator is

$$\hat{n}_{b,i} = \hat{b}_i^\dagger \hat{b}_i = \sum_{n=0}^{n_{\mathrm{ph,max}}} n\, |n\rangle_i \langle n|,$$

and the displacement operator is

$$\hat{x}_i = \hat{b}_i + \hat{b}_i^\dagger = \sum_{n=0}^{n_{\mathrm{ph,max}}-1} \sqrt{n+1}\Big(|n+1\rangle_i \langle n| + |n\rangle_i \langle n+1|\Big).$$

4.2 Binary encoding

In binary encoding,

$$q_{\mathrm{pb}} = \max\{1, \lceil \log_2(n_{\mathrm{ph,max}} + 1) \rceil\}.$$

For site `i`, the phonon qubit block starts at

$$q_{\mathrm{base}}(i) = 2L + i\, q_{\mathrm{pb}}.$$

So the local phonon qubits are

$$q_{\mathrm{base}}(i), q_{\mathrm{base}}(i) + 1, \dots, q_{\mathrm{base}}(i) + q_{\mathrm{pb}} - 1.$$

The binary implementation pads the local `d\times d` oscillator matrix into the `2^{q_pb}` qubit space and then decomposes it in the local Pauli basis:

$$\hat{M}_i^{\mathrm{pad}} = \sum_{\alpha \in \{I,X,Y,Z\}^{\otimes q_{\mathrm{pb}}}} \frac{1}{2^{q_{\mathrm{pb}}}} \mathrm{Tr}\big(P_\alpha^\dagger \hat{M}_i^{\mathrm{pad}}\big) P_\alpha,$$

with `M = b, b†, n, x`.

That explicit Pauli decomposition is what `boson_local_operator_pauli_decomp(...)` and `boson_operator(...)` implement.

4.3 Unary encoding

In unary encoding,

$$q_{\mathrm{pb}} = n_{\mathrm{ph,max}} + 1.$$

The qubit corresponding to local level `n` at site `i` is

$$q(i,n) = 2L + i(n_{\mathrm{ph,max}} + 1) + n.$$

The unary one-hot projector is

$$\hat{n}_{i,n} = |1\rangle\langle 1|_{q(i,n)} = \frac{I - Z_{q(i,n)}}{2}.$$

Therefore the unary phonon number operator is already explicit in single-qubit `Z` primitives:

$$\hat{n}_{b,i} = \sum_{n=0}^{n_{\mathrm{ph,max}}} n\, \hat{n}_{i,n} = \sum_{n=0}^{n_{\mathrm{ph,max}}} n\, \frac{I - Z_{q(i,n)}}{2}.$$

Define

$$\sigma_q^+ = \frac{X_q + iY_q}{2}, \qquad \sigma_q^- = \frac{X_q - iY_q}{2}.$$

Then the unary ladder operators are

$$\hat{b}_i^\dagger = \sum_{n=0}^{n_{\mathrm{ph,max}}-1} \sqrt{n+1}\, \sigma^+_{q(i,n)} \sigma^-_{q(i,n+1)},$$

$$\hat{b}_i = \sum_{n=0}^{n_{\mathrm{ph,max}}-1} \sqrt{n+1}\, \sigma^-_{q(i,n)} \sigma^+_{q(i,n+1)}.$$

So the unary displacement operator is

$$\hat{x}_i = \hat{b}_i + \hat{b}_i^\dagger = \frac{1}{2} \sum_{n=0}^{n_{\mathrm{ph,max}}-1} \sqrt{n+1} \Big(X_{q(i,n)} X_{q(i,n+1)} + Y_{q(i,n)} Y_{q(i,n+1)} \Big).$$

4.4 Phonon vacuum

4.4.1 Binary vacuum

The binary phonon vacuum is the all-zero phonon register:

$$|\mathrm{vac}_{\mathrm{ph}}\rangle_{\mathrm{binary}} = |0\cdots 0\rangle.$$

4.4.2 Unary vacuum

The unary phonon vacuum is one-hot at `n=0` for each site, so each site block contributes

$$0^{q_{\mathrm{pb}}-1}1$$

in printed `q_(N_q-1)...q_0` order.

Implemented surfaces:

- `src/quantum/hubbard_latex_python_pairs.py`
  - `boson_qubits_per_site`
  - `phonon_qubit_indices_for_site`
  - `boson_operator`
  - `boson_number_operator`
  - `boson_displacement_operator`
- `src/quantum/hartree_fock_reference_state.py`
  - `_phonon_vacuum_bitstring`

5. Hubbard Hamiltonian by Explicit Substitution

5.1 Kinetic term

The Hubbard hopping term is

$$\hat{H}_t = -J \sum_{\langle i,j\rangle,\sigma} \left(\hat{c}^\dagger_{i\sigma}\hat{c}_{j\sigma} + \hat{c}^\dagger_{j\sigma}\hat{c}_{i\sigma}\right).$$

Now substitute

$$p_i = p(i,\sigma), \qquad p_j = p(j,\sigma), \qquad p_< = \min\{p_i, p_j\}, \qquad p_> = \max\{p_i, p_j\}.$$

After inserting the JW ladder primitives and collecting the Hermitian pair, the repository's hopping generator becomes

$$\hat{c}^\dagger_{p_i}\hat{c}_{p_j} + \hat{c}^\dagger_{p_j}\hat{c}_{p_i} = \frac{1}{2}\Big(X_{p_>}Z_{p_>-1}\cdots Z_{p_<+1}X_{p_<} + Y_{p_>}Z_{p_>-1}\cdots Z_{p_<+1}Y_{p_<}\Big).$$

So the fully substituted kinetic term is

$$\hat{H}_t = -\frac{J}{2}\sum_{\langle i,j\rangle,\sigma}\Big(X_{p_>(i,j,\sigma)}Z\cdots ZX_{p_<(i,j,\sigma)} + Y_{p_>(i,j,\sigma)}Z\cdots ZY_{p_<(i,j,\sigma)}\Big).$$

5.2 Onsite interaction

The onsite interaction starts as

$$\hat{H}_U = U\sum_i \hat{n}_{i\uparrow}\hat{n}_{i\downarrow}.$$

Substitute the number primitives:

$$\hat{H}_U = U\sum_i \frac{I - Z_{p_{i\uparrow}}}{2}\frac{I - Z_{p_{i\downarrow}}}{2}.$$

Multiply explicitly:

$$\hat{H}_U = \frac{U}{4}\sum_i\Big(I - Z_{p_{i\uparrow}} - Z_{p_{i\downarrow}} + Z_{p_{i\uparrow}}Z_{p_{i\downarrow}}\Big).$$

5.3 Static potential term

The static site-potential term is

$$\hat{H}_v = -\sum_{i,\sigma} v_i \hat{n}_{i\sigma}.$$

Substitute `\hat n_{i\sigma}=(I-Z_{p(i,\sigma)})/2`:

$$\hat{H}_v = -\sum_{i,\sigma} v_i \frac{I - Z_{p(i,\sigma)}}{2} = -\frac{1}{2}\sum_{i,\sigma} v_i I + \frac{1}{2}\sum_{i,\sigma} v_i Z_{p(i,\sigma)}.$$

If the potential is uniform, `v_i=dv`, then this becomes

$$\hat{H}_v = -L\, dv\, I + \frac{dv}{2}\sum_{i,\sigma} Z_{p(i,\sigma)}.$$

5.4 Fully substituted Hubbard Hamiltonian

The full Hubbard Hamiltonian is therefore

$$\hat{H}_{\text{Hub}} = -\frac{J}{2}\sum_{\langle i,j\rangle,\sigma}\Big(X_{p_>} Z \cdots Z X_{p_<} + Y_{p_>} Z \cdots Z Y_{p_<}\Big) + \frac{U}{4}\sum_i \Big(I - Z_{p_{i\uparrow}} - Z_{p_{i\downarrow}} + Z_{p_{i\uparrow}} Z_{p_{i\downarrow}}\Big) - \frac{1}{2}\sum_{i,\sigma} v_i I + \frac{1}{2}\sum_{i,\sigma} v_i Z_{p(i,\sigma)}.$$

Implemented surfaces:

- `src/quantum/hubbard_latex_python_pairs.py`
  - `build_hubbard_kinetic`
  - `build_hubbard_onsite`
  - `build_hubbard_potential`
  - `build_hubbard_hamiltonian`
- `src/quantum/vqe_latex_python_pairs.py`
  - `hubbard_hop_term`
  - `hubbard_onsite_term`
  - `hubbard_potential_term`

6. Hubbard-Holstein Hamiltonian by Explicit Substitution

6.1 Phonon energy

The phonon energy is

$$\hat{H}_{\text{ph}} = \omega_0 \sum_i \left(\hat{n}_{b,i} + \frac{1}{2} I\right).$$

6.1.1 Unary explicit form

Insert the unary number operator:

$$\hat{H}_{\text{ph}}^{\text{unary}} = \omega_0 \sum_i \left(\sum_{n=0}^{n_{\text{ph,max}}} n\, \frac{I - Z_{q(i,n)}}{2} + \frac{1}{2} I\right).$$

6.1.2 Binary explicit form

Insert the binary Pauli decomposition of `\hat n_{b,i}`:

$$\hat{n}_{b,i} = \sum_\alpha \frac{1}{2^{q_{\text{pb}}}} \operatorname{Tr}(P_\alpha^\dagger \hat{n}_{b,i}^{\text{pad}}) P_{i,\alpha}.$$

So the binary phonon energy is

$$\hat{H}_{\text{ph}}^{\text{binary}} = \omega_0 \sum_i \left(\sum_\alpha \frac{1}{2^{q_{\text{pb}}}} \operatorname{Tr}(P_\alpha^\dagger \hat{n}_{b,i}^{\text{pad}}) P_{i,\alpha} + \frac{1}{2} I\right).$$

6.2 Electron-phonon coupling

The Holstein coupling starts as

$$\hat{H}_g = g \sum_i \hat{x}_i (\hat{n}_i - I).$$

Now substitute the explicit site density from Section 3.3:

$$\hat{n}_i - I = \Big(I - \frac{1}{2}(Z_{p_{i\uparrow}} + Z_{p_{i\downarrow}}) \Big) - I = -\frac{1}{2}(Z_{p_{i\uparrow}} + Z_{p_{i\downarrow}}).$$

So the coupling becomes

$$\hat{H}_g = -\frac{g}{2} \sum_i \hat{x}_i (Z_{p_{i\uparrow}} + Z_{p_{i\downarrow}}).$$

This is the most important substitution in the HH layer: the fermion density shift `\hat n_i-I` is not left abstract; it is reduced all the way to explicit `Z` operators.

6.2.1 Unary explicit form

Insert the unary displacement operator:

$$\hat{x}_i = \frac{1}{2} \sum_{n=0}^{n_{\mathrm{ph,max}}-1} \sqrt{n+1} \Big(X_{q(i,n)} X_{q(i,n+1)} + Y_{q(i,n)} Y_{q(i,n+1)} \Big).$$

Therefore

$$\hat{H}_g^{\mathrm{unary}} = -\frac{g}{4} \sum_i \left[\sum_{n=0}^{n_{\mathrm{ph,max}}-1} \sqrt{n+1} \Big(X_{q(i,n)} X_{q(i,n+1)} + Y_{q(i,n)} Y_{q(i,n+1)} \Big) \right] (Z_{p_{i\uparrow}} + Z_{p_{i\downarrow}}).$$

6.2.2 Binary explicit form

Insert the binary Pauli decomposition of `\hat x_i`:

$$\hat{x}_i = \sum_\alpha \frac{1}{2^{q_{\mathrm{pb}}}} \mathrm{Tr}(P_\alpha^\dagger \hat{x}_i^{\mathrm{pad}}) P_{i,\alpha}.$$

Then

$$\hat{H}_g^{\mathrm{binary}} = -\frac{g}{2} \sum_i \left[\sum_\alpha \frac{1}{2^{q_{\mathrm{pb}}}} \mathrm{Tr}(P_\alpha^\dagger \hat{x}_i^{\mathrm{pad}}) P_{i,\alpha} \right] (Z_{p_{i\uparrow}} + Z_{p_{i\downarrow}}).$$

6.3 Time-dependent density drive

The implemented HH drive builder uses the physical increment

$$\delta v_i(t) = v_i(t) - v_{0,i}$$

and constructs

$$\hat{H}_{\mathrm{drive}}(t) = \sum_{i,\sigma} \delta v_i(t) \hat{n}_{i\sigma}.$$

Now substitute the number operator explicitly:

$$\hat{H}_{\mathrm{drive}}(t) = \sum_{i,\sigma} \delta v_i(t) \frac{I - Z_{p(i,\sigma)}}{2} = \frac{1}{2} \sum_{i,\sigma} \delta v_i(t) I - \frac{1}{2} \sum_{i,\sigma} \delta v_i(t) Z_{p(i,\sigma)}.$$

Because there are two spins per site, the identity contribution can also be written as

$$\frac{1}{2} \sum_{i,\sigma} \delta v_i(t) I = \sum_i \delta v_i(t) I.$$

The implementation route in `build_hubbard_holstein_drive(...)` is explicit:

- `build_hubbard_potential(...)` assumes `H_v=-\sum_{i,\sigma} v_i n_{i\sigma}`,
- therefore the HH drive builder passes `v_for_existing(i)=-\delta v_i(t)`.

6.3.1 Implemented density-drive waveform surface

The runtime density-drive helper in `src/quantum/drives_time_potential.py` uses the scalar waveform

$$f(\tau) = A\sin(\omega\tau + \phi)\exp\left(-\frac{\tau^2}{2\bar{t}^2}\right),$$

with the sampled time

$$\tau = t + t_0.$$

So the site-resolved potential surface is

$$v_i(t) = s_i f(t + t_0),$$

where the spatial weights are

$$s_i \in \{(-1)^i,\ [+1, -1] \text{ for } L = 2, \text{ custom user weights}\}.$$

The runtime Pauli-coefficient map is then

$$\Delta c[Z_{p(i,\sigma)}](t) = -\frac{1}{2}v_i(t),$$

and, when requested,

$$\Delta c[I](t) = \sum_i v_i(t).$$

6.4 Two implemented HH assembly surfaces

There are two distinct implemented HH assembly surfaces and this manuscript states both instead of pretending they are identical.

6.4.1 Core Hamiltonian builder surface

`build_hubbard_holstein_hamiltonian(...)` assembles

$$\hat{H}_{\mathrm{HH,core}}(t) = \hat{H}_t + \hat{H}_U + \hat{H}_{\mathrm{ph}} + \hat{H}_g + \hat{H}_{\mathrm{drive}}(t),$$

with the Hubbard part built using `v=None`.

6.4.2 Variational HH ansatz surface

The HH ansatz classes in `src/quantum/vqe_latex_python_pairs.py` expose the more general grouped surface

$$\hat{H}_{\mathrm{HH,ansatz}}(t) = \hat{H}_t + \hat{H}_U + \hat{H}_v + \hat{H}_{\mathrm{ph}} + \hat{H}_g + \hat{H}_{\mathrm{drive}}(t),$$

where the static fermion potential `\hat H_v` may be included separately.

6.5 Fully substituted HH master form

Combining the substitutions above gives the primitive-closed HH expression

$$
\begin{aligned}
\hat{H}_{\mathrm{HH}}(t) = &-\frac{J}{2}\sum_{\langle i,j\rangle,\sigma}\Big(X_{p_>}Z\cdots ZX_{p_<} + Y_{p_>}Z\cdots ZY_{p_<}\Big) \\
&+\frac{U}{4}\sum_{i}\Big(I - Z_{p_{i\uparrow}} - Z_{p_{i\downarrow}} + Z_{p_{i\uparrow}}Z_{p_{i\downarrow}}\Big) \\
&-\frac{1}{2}\sum_{i,\sigma} v_i I + \frac{1}{2}\sum_{i,\sigma} v_i Z_{p(i,\sigma)} \\
&+\omega_0\sum_{i}\left(\hat{n}_{b,i} + \frac{1}{2}I\right) \\
&-\frac{g}{2}\sum_{i}\hat{x}_i(Z_{p_{i\uparrow}} + Z_{p_{i\downarrow}}) \\
&+\frac{1}{2}\sum_{i,\sigma}\delta v_i(t) I - \frac{1}{2}\sum_{i,\sigma}\delta v_i(t) Z_{p(i,\sigma)}.
\end{aligned}
$$

At this point only `\hat n_{b,i}` and `\hat x_i` remain as boson primitives, and those are already given explicitly in binary and unary form in Section 4.

Implemented surfaces:

- `src/quantum/hubbard_latex_python_pairs.py`
  - `build_holstein_phonon_energy`
  - `build_holstein_coupling`
  - `build_hubbard_holstein_drive`
  - `build_hubbard_holstein_hamiltonian`
- `src/quantum/drives_time_potential.py`
  - `GaussianSinusoidSitePotential`
  - `DensityDriveTemplate`
  - `TimeDependentOnsiteDensityDrive`
  - `build_gaussian_sinusoid_density_drive`

7. Reference States and Exact Sector ED

7.1 Fermionic Hartree-Fock determinant

The repository's Hartree-Fock reference fills the first `n_alpha` up-spin orbitals and first `n_beta` down-spin orbitals in the chosen ordering.

If the occupied qubit set is `Q_occ`, then the HF bitstring in `q_(N_q-1)...q_0` order is the string with

$$\mathrm{bit}(q) = 1 \iff q \in Q_{\mathrm{occ}}.$$

The corresponding basis index is

$$k_{\mathrm{HF}} = \sum_{q\in Q_{\mathrm{occ}}} 2^q.$$

The half-filled default used by the HH reference helper is

$$(N_\uparrow, N_\downarrow) = \left(\left\lceil \frac{L}{2}\right\rceil, \left\lfloor \frac{L}{2}\right\rfloor\right).$$

7.2 Hubbard-Holstein reference state

The HH reference state is

$$|\psi_{\mathrm{ref}}^{\mathrm{HH}}\rangle = |\mathrm{vac}_{\mathrm{ph}}\rangle \otimes |\Phi_{\mathrm{HF}}\rangle.$$

The printed basis label is built exactly as

$$\texttt{full_bitstring} = \texttt{phonon_vacuum_bitstring} + \texttt{hf_fermion_bitstring}.$$

So the phonon block is displayed on the left and the fermion block on the right.

7.3 Exact HH sector basis

The exact ED surface in `src/quantum/ed_hubbard_holstein.py` is independent of the Pauli-polynomial builder. It constructs a basis of physical states

$$|\text{fermion bits}; n_0, n_1, \dots, n_{L-1}\rangle$$

subject to

- fixed fermion sector `(N_up, N_dn)`,
- unrestricted local phonon occupations inside the truncation `0\le n_i\le n_ph_max`.

Binary and unary encodings change only the map from a physical state to the computational basis index:

7.3.1 Binary index map

If the fermion basis bits form the integer `f_bits`, then

$$\mathrm{index}_{\mathrm{binary}} = f_{\mathrm{bits}} + \sum_{i=0}^{L-1} n_i \, 2^{2L+iq_{\mathrm{pb}}}.$$

This is the compact register interpretation implemented by `encode_state_to_qubit_index(...)`.

7.3.2 Unary index map

If the same physical state uses unary phonon encoding, then the computational-basis index is built by setting one-hot qubits `q(i,n_i)` in addition to the fermion bits. The physical matrix elements are unchanged; only the encoded index changes.

7.4 Exact HH matrix elements

The exact ED builder constructs

$$\hat{H}_{\mathrm{HH}} = \hat{H}_t + \hat{H}_U + \hat{H}_{\mathrm{ph}} + \hat{H}_g + \hat{H}_{\mathrm{drive}}$$

directly in the occupation basis.

7.4.1 Diagonal elements

For a basis state with fermion occupations `n_{i\uparrow}, n_{i\downarrow}` and phonon occupations `n_i^{\mathrm{ph}}`, the diagonal contribution is

$$\sum_i \Big[U n_{i\uparrow} n_{i\downarrow} + \delta v_i (n_{i\uparrow} + n_{i\downarrow}) + \omega_0 n_i^{\mathrm{ph}} \Big] + \frac{\omega_0 L}{2}$$

when `include_zero_point=True`.

7.4.2 Hopping matrix elements

If `\hat c_p^{\dagger}\hat c_q` is allowed on the fermion bitstring, then the off-diagonal matrix element is

$$-J \times (\text{fermionic sign from JW parity}).$$

The helper `_apply_cdag_c(...)` computes the sign explicitly by counting occupied qubits below the action site.

7.4.3 Electron-phonon matrix elements

If the site density gives the prefactor

$$g(n_i - 1),$$

then `\hat x_i=\hat b_i+\hat b_i^{\dagger}` contributes

$$\langle n_i + 1|\hat{x}_i|n_i\rangle = \sqrt{n_i + 1}, \qquad \langle n_i - 1|\hat{x}_i|n_i\rangle = \sqrt{n_i}.$$

So the ED builder inserts off-diagonal matrix elements

$$g(n_i - 1)\sqrt{n_i + 1} \quad \text{or} \quad g(n_i - 1)\sqrt{n_i}$$

for phonon raising or lowering respectively.

Implemented surfaces:

- `src/quantum/hartree_fock_reference_state.py`
- `src/quantum/ed_hubbard_holstein.py`

8. Statevector Action, Expectation, and Exponential Primitives

8.1 Basis-state primitive

The statevector primitive `basis_state(N_q, bitstring)` places amplitude `1` at

$$\text{index} = \text{int}(\texttt{bitstring}, 2),$$

with the bitstring already in `q_(N_q-1)...q_0` order.

8.2 Explicit Pauli-word action

Let the Pauli word be

$$P = \sigma_{N_q-1} \cdots \sigma_1 \sigma_0, \qquad \sigma_q \in \{e, x, y, z\}.$$

Define the bit-flip set

$$F(P) = \{q \mid \sigma_q \in \{x, y\}\},$$

and the phase on basis index `k` with bits `b_q(k)` as

$$\phi_P(k) = \prod_{q:\sigma_q=z} (-1)^{b_q(k)} \prod_{q:\sigma_q=y} i\,(-1)^{b_q(k)}.$$

Then the implemented Pauli action is

$$P|k\rangle = \phi_P(k)\,|k \oplus \chi_{F(P)}\rangle,$$

so on amplitudes,

$$(P\psi)_{k\oplus\chi_{F(P)}} = \phi_P(k)\,\psi_k.$$

This is exactly what `apply_pauli_string(...)` does by looping over qubits and applying `x`, `y`, and `z` cases with the repository's rightmost-`q_0` convention.

8.3 Expectation values

If

$$\hat{H} = \sum_j h_j P_j,$$

then the variational energy is

$$E(\psi) = \langle\psi|\hat{H}|\psi\rangle = \sum_j h_j \langle\psi|P_j|\psi\rangle.$$

`expval_pauli_polynomial(...)` evaluates this term by term.

The compiled one-apply backend instead computes

$$E(\psi) = \mathrm{Re}\langle\psi|\hat{H}\psi\rangle,$$

after compiling the polynomial action once.

8.4 Pauli rotations

For a single Pauli word `P` with `P^2=I`, the implemented rotation primitive is

$$R_P(\varphi) = \exp\left(-i\frac{\varphi}{2}P\right) = \cos\left(\frac{\varphi}{2}\right) I - i\sin\left(\frac{\varphi}{2}\right) P.$$

So the updated state is

$$R_P(\varphi)|\psi\rangle = \cos\left(\frac{\varphi}{2}\right)|\psi\rangle - i\sin\left(\frac{\varphi}{2}\right) P|\psi\rangle.$$

8.5 Exponential of a Pauli polynomial

If

$$\hat{H} = \sum_j h_j P_j,$$

then the code applies the first-order ordered product

$$\exp(-i\theta\hat{H})|\psi\rangle \approx \prod_j \exp(-i\theta h_j P_j)|\psi\rangle.$$

Because `apply_pauli_rotation(...)` expects the angle `\varphi` in

$$\exp\left(-i\frac{\varphi}{2}P\right),$$

`apply_exp_pauli_polynomial(...)` uses

$$\varphi_j = 2\theta h_j.$$

So the implemented ordered update is

$$|\psi_{\mathrm{out}}\rangle \approx \prod_j \left[\cos(\theta h_j)I - i\sin(\theta h_j)P_j\right]|\psi_{\mathrm{in}}\rangle,$$

after optionally dropping identity terms and sorting the Pauli words deterministically.

8.6 Exact sector energy target

The exact HH target used by the hardcoded VQE surfaces is a sector-filtered exact energy, not an unrestricted full-Hilbert minimum. Only the fermionic particle numbers are fixed; the phonon qubits are left unconstrained. This is what `exact_ground_energy_sector_hh(...)` and the ED basis logic provide.

Write the total Hilbert space as

$$\mathcal{H} = \mathcal{H}_{\mathrm{ferm}} \otimes \mathcal{H}_{\mathrm{ph}},$$

with qubit layout

$$[\,2L \text{ fermion qubits} \mid L \cdot q_{\mathrm{pb}} \text{ phonon qubits}\,].$$

Let $\mathcal{J}_\alpha$ and $\mathcal{J}_\beta$ denote the fermion-qubit index sets for spin-up and spin-down. In the two supported orderings,

$$\mathcal{J}_\alpha = \begin{cases} \{0,\dots,L-1\}, & \texttt{blocked}, \\ \{0,2,\dots,2L-2\}, & \texttt{interleaved}, \end{cases} \qquad \mathcal{J}_\beta = \begin{cases} \{L,\dots,2L-1\}, & \texttt{blocked}, \\ \{1,3,\dots,2L-1\}, & \texttt{interleaved}. \end{cases}$$

Using the qubit number operator

$$\hat{n}_q = \frac{I - Z_q}{2},$$

define the fermion-number operators

$$\hat{N}_\alpha = \sum_{q \in \mathcal{J}_\alpha} \hat{n}_q, \qquad \hat{N}_\beta = \sum_{q \in \mathcal{J}_\beta} \hat{n}_q.$$

For target counts (N_α, N_β), the HH sector is

$$\mathcal{H}_{N_\alpha, N_\beta} = \left\{ |\psi\rangle \in \mathcal{H} : \hat{N}_\alpha |\psi\rangle = N_\alpha |\psi\rangle,\ \hat{N}_\beta |\psi\rangle = N_\beta |\psi\rangle \right\},$$

with no restriction on the phonon occupation.

Equivalently, in the computational basis $|f, p\rangle$ with fermion bits $f = (f_q)$ and phonon bits p, the projector onto this sector is

$$\hat{\Pi}_{N_\alpha, N_\beta} = \sum_{f,p} \delta_{\sum_{q \in \mathcal{J}_\alpha} f_q,\, N_\alpha}\, \delta_{\sum_{q \in \mathcal{J}_\beta} f_q,\, N_\beta}\, |f, p\rangle\langle f, p|.$$

So for an arbitrary state

$$|\psi\rangle = \sum_{f,p} c_{f,p} |f, p\rangle,$$

the sector-filtered component is

$$\hat{\Pi}_{N_\alpha, N_\beta} |\psi\rangle = \sum_{f,p} \delta_{\sum_{q \in \mathcal{J}_\alpha} f_q,\, N_\alpha}\, \delta_{\sum_{q \in \mathcal{J}_\beta} f_q,\, N_\beta}\, c_{f,p} |f, p\rangle.$$

If normalization is needed, use

$$|\psi_{N_\alpha, N_\beta}\rangle = \frac{\hat{\Pi}_{N_\alpha, N_\beta} |\psi\rangle}{\sqrt{\langle \psi | \hat{\Pi}_{N_\alpha, N_\beta} | \psi\rangle}}.$$

The exact sector energy target is then the lowest eigenvalue of the projected Hamiltonian,

$$E_0^{(N_\alpha, N_\beta)} = \min_{\substack{|\psi\rangle \in \mathcal{H}_{N_\alpha, N_\beta} \\ \langle \psi | \psi \rangle = 1}} \langle \psi | \hat{H} | \psi \rangle = \lambda_{\min}\left(\hat{\Pi}_{N_\alpha, N_\beta} \hat{H} \hat{\Pi}_{N_\alpha, N_\beta} \right).$$

In matrix form, the implementation builds the full Hamiltonian matrix M, keeps only the basis indices satisfying the two fermion-count constraints, forms the submatrix

$$M_{\text{sector}} = M[\mathcal{B}_{N_\alpha, N_\beta}, \mathcal{B}_{N_\alpha, N_\beta}],$$

and returns

$$E_0^{(N_\alpha, N_\beta)} = \lambda_{\min}(M_{\text{sector}}).$$

Implemented surfaces:

- `src/quantum/vqe_latex_python_pairs.py`
  - `basis_state`
  - `apply_pauli_string`
  - `expval_pauli_polynomial`
  - `expval_pauli_polynomial_one_apply`
  - `apply_pauli_rotation`
  - `apply_exp_pauli_polynomial`
  - `exact_ground_energy_sector_hh`

9. Current Hardcoded Fixed VQE Ansatz Families

The user-facing fixed ansatz choices in `pipelines/hardcoded/hubbard_pipeline.py` are

- `uccsd`,
- `hva`,
- `hh_hva`,
- `hh_hva_tw`,
- `hh_hva_ptw`.

The ADAPT pool/replay-family surfaces are documented separately in Sections 10 and 13.

9.1 Hubbard layerwise ansatz: `hva`

For the static Hubbard problem, the fixed layerwise ansatz groups the Hamiltonian into

1. hopping,
2. onsite interaction,
3. static potential if present.

Write

$$\hat{H}_{\mathrm{Hub}} = \hat{H}_t + \hat{H}_U + \hat{H}_v,$$

with

$$\hat{H}_t = -t \sum_{\langle i,j\rangle,\sigma} \left(\hat{c}^\dagger_{i\sigma}\hat{c}_{j\sigma} + \hat{c}^\dagger_{j\sigma}\hat{c}_{i\sigma} \right),$$

$$\hat{H}_U = U \sum_i \hat{n}_{i\uparrow}\hat{n}_{i\downarrow},$$

$$\hat{H}_v = \sum_{i,\sigma} v_i \hat{n}_{i\sigma}.$$

For layer `\ell`, if

$$\hat{H}_{\mathrm{group}} = \sum_{P\in\mathrm{split}(\hat{H}_{\mathrm{group}})} h_P P,$$

then the implemented Hubbard layerwise unitary is

$$\hat{U}^{(\ell)}_{\text{Hub-layer}} = \prod_{P\in\mathrm{split}(\hat{H}_t)} e^{-i\theta_t^{(\ell)} h_P P} \prod_{P\in\mathrm{split}(\hat{H}_U)} e^{-i\theta_U^{(\ell)} h_P P} \prod_{P\in\mathrm{split}(\hat{H}_v)} e^{-i\theta_v^{(\ell)} h_P P}.$$

So all split Pauli terms inside one physical group share one parameter per layer.

9.2 Hubbard hardcoded UCCSD surface: `uccsd`

Relative to the Hartree-Fock occupied/virtual partition, the primitive Hermitian single- and double-excitation generators are

$$G^{(\sigma)}_{i\to a} = i\left(\hat{c}^\dagger_{a\sigma}\hat{c}_{i\sigma} - \hat{c}^\dagger_{i\sigma}\hat{c}_{a\sigma} \right),$$

$$G^{(\sigma,\sigma')}_{ij\to ab} = i\left(\hat{c}^\dagger_{a\sigma}\hat{c}^\dagger_{b\sigma'}\hat{c}_{j\sigma'}\hat{c}_{i\sigma} - \hat{c}^\dagger_{i\sigma}\hat{c}^\dagger_{j\sigma'}\hat{c}_{b\sigma'}\hat{c}_{a\sigma} \right).$$

If `\mathcal S` and `\mathcal D` denote the generated single and double excitation sets, then the fixed hardcoded CLI surface `--vqe-ansatz uccsd` dispatches to the **layerwise** form

$$\hat{U}^{(\ell)}_{\text{UCCSD-layer}} = \prod_{G\in\mathcal{S}} e^{-i\theta_s^{(\ell)} G} \prod_{G\in\mathcal{D}} e^{-i\theta_d^{(\ell)} G}.$$

So the live fixed VQE surface shares one parameter across all singles and one across all doubles per layer.

The repo also contains the lower-level base class `HardcodedUCCSDAnsatz`,

$$\hat{U}_{\text{UCCSD-base}}(\theta) = \prod_k e^{-i\theta_k G_k},$$

with one independent parameter per excitation generator. ADAPT pool builders reuse those base generators, but the fixed hardcoded VQE CLI chooses the shared-parameter layerwise surface above.

9.3 HH layerwise ansatz: `hh_hva`

The layerwise HH ansatz groups the Hamiltonian into physical sectors:

1. hopping,
2. onsite interaction,
3. static potential if present,
4. phonon energy,
5. electron-phonon coupling,
6. drive if present.

For layer `\ell`, write the split Pauli terms of each physical group as

$$\hat{H}_{\text{group}} = \sum_{P \in \text{split}(\hat{H}_{\text{group}})} h_P P.$$

Then the implemented layerwise unitary is

$$\hat{U}^{(\ell)}_{\text{layer}} = \prod_{P \in \text{split}(\hat{H}_t)} e^{-i\theta^{(\ell)}_t h_P P} \prod_{P \in \text{split}(\hat{H}_U)} e^{-i\theta^{(\ell)}_U h_P P} \prod_{P \in \text{split}(\hat{H}_v)} e^{-i\theta^{(\ell)}_v h_P P} \prod_{P \in \text{split}(\hat{H}_{\text{ph}})} e^{-i\theta^{(\ell)}_{\text{ph}} h_P P} \prod_{P \in \text{split}(\hat{H}_g)} e^{-i\theta^{(\ell)}_g h_P P} \prod_{P \in \text{split}(\hat{H}_{\text{drive}})} e^{-i\theta^{(\ell)}_{\text{driv}}}$$

The crucial point is that **all split Pauli terms inside one physical group share one parameter per layer**.

9.4 HH Pauli-termwise ansatz: `hh_hva_tw`

The Pauli-termwise HH ansatz removes that sharing. Every single split Pauli term gets its own parameter:

$$\hat{U}^{(\ell)}_{\text{tw}} = \prod_{P \in \text{split}(\hat{H}_t)} e^{-i\theta^{(\ell)}_{t,P} h_P P} \prod_{P \in \text{split}(\hat{H}_U)} e^{-i\theta^{(\ell)}_{U,P} h_P P} \prod_{P \in \text{split}(\hat{H}_v)} e^{-i\theta^{(\ell)}_{v,P} h_P P} \prod_{P \in \text{split}(\hat{H}_{\text{ph}})} e^{-i\theta^{(\ell)}_{\text{ph},P} h_P P} \prod_{P \in \text{split}(\hat{H}_g)} e^{-i\theta^{(\ell)}_{g,P} h_P P} \prod_{P \in \text{split}(\hat{H}_{\text{drive}})} e^{-i}$$

This is more expressive, but the split single-Pauli factors need not preserve the fermion sector individually.

9.5 HH physical-termwise ansatz: `hh_hva_ptw`

The physical-termwise HH ansatz keeps one parameter per physical generator before Pauli splitting.

Its Hamiltonian contract is

$$\hat{H}_{\text{HH}}(t) = \hat{H}_t + \hat{H}_U + \hat{H}_{\text{ph}} + \hat{H}_g + \hat{H}_{\text{drive}}(t),$$

with

$$\hat{H}_t = -J \sum_{\langle i,j \rangle, \sigma} \left(\hat{c}^\dagger_{i\sigma} \hat{c}_{j\sigma} + \hat{c}^\dagger_{j\sigma} \hat{c}_{i\sigma} \right),$$

$$\hat{H}_U = U \sum_i \hat{n}_{i\uparrow} \hat{n}_{i\downarrow},$$

$$\hat{H}_{\text{ph}} = \omega_0 \sum_i \left(\hat{n}_{b,i} + \frac{1}{2} \right),$$

$$\hat{H}_g = g \sum_i \hat{x}_i (\hat{n}_i - \mathbb{1}), \qquad \hat{n}_i = \hat{n}_{i\uparrow} + \hat{n}_{i\downarrow},\ \hat{x}_i = b_i + b_i^\dagger,$$

$$\hat{H}_{\text{drive}}(t) = \sum_{i,\sigma} (v_i(t) - v_{0,i}) \hat{n}_{i\sigma}.$$

For one layer,

$$\hat{U}_{\mathrm{ptw}}^{(\ell)} = \prod_{\langle i,j\rangle,\sigma} e^{-i\theta_{ij\sigma}^{(\ell)}\hat{H}_{ij\sigma}^{(t)}} \prod_i e^{-i\theta_{U,i}^{(\ell)}\hat{H}_i^{(U)}} \prod_{i,\sigma} e^{-i\theta_{v,i\sigma}^{(\ell)}\hat{H}_{i\sigma}^{(v)}} \prod_i e^{-i\theta_{\mathrm{ph},i}^{(\ell)}\omega_0\hat{n}_{b,i}} \prod_i e^{-i\theta_{g,i}^{(\ell)}g\hat{x}_i(\hat{n}_i - I)} \prod_{i,\sigma} e^{-i\theta_{d,i\sigma}^{(\ell)}\delta v_i(t)\hat{n}_{i\sigma}}.$$

Here

$$\hat{H}_{ij\sigma}^{(t)} = -J\left(\hat{c}_{i\sigma}^{\dagger}\hat{c}_{j\sigma} + \hat{c}_{j\sigma}^{\dagger}\hat{c}_{i\sigma}\right),$$
$$\hat{H}_i^{(U)} = U\hat{n}_{i\uparrow}\hat{n}_{i\downarrow},$$
$$\hat{H}_{i\sigma}^{(v)} = -v_i\hat{n}_{i\sigma}.$$

In the implemented HH PTW path (`HubbardHolsteinPhysicalTermwiseAnsatz`), generator selection is done on physical sectors (`H_t`, `H_U`, `H_{\mathrm{ph}}`, `H_g`, optional `H_{\mathrm{drive}}`), then each sector is instantiated through the same `hubbard_latex_python_pairs.py` builders that immediately materialize mapped Pauli polynomials via `hubbard_*_term`, `jw_number_operator`, `boson_*` helpers. That means selection is **before Pauli-term splitting**, but each selected generator is already in mapped Pauli form by construction.

This ansatz is sector-preserving in fermion space because each physical generator preserves fermion number before Pauli splitting.

9.6 Reference-state and ansatz dispatch in the hardcoded pipeline

For `problem="hubbard"`, the pipeline uses the Hartree-Fock basis state

$$|\psi_{\mathrm{ref}}\rangle = |\Phi_{\mathrm{HF}}\rangle.$$

For `problem="hh"`, it uses

$$|\psi_{\mathrm{ref}}\rangle = |\mathrm{vac}_{\mathrm{ph}}\rangle \otimes |\Phi_{\mathrm{HF}}\rangle$$

via `hubbard_holstein_reference_state(...)`.

The live dispatch is

- `uccsd` -> `HardcodedUCCSDLayerwiseAnsatz`,
- `hva` -> `HubbardLayerwiseAnsatz`,
- `hh_hva` -> `HubbardHolsteinLayerwiseAnsatz`,
- `hh_hva_tw` -> `HubbardHolsteinTermwiseAnsatz`,
- `hh_hva_ptw` -> `HubbardHolsteinPhysicalTermwiseAnsatz`.

Implemented surfaces:

- `src/quantum/vqe_latex_python_pairs.py`
- `pipelines/hardcoded/hubbard_pipeline.py`

10. PAOP Families, ADAPT Selection, and Staged Continuation

10.1 Primitive PAOP ingredients

The PAOP layer is implemented in `src/quantum/operator_pools/polaron_paop.py`.

10.1.1 Shifted density

If the total electron count is `N_e`, then the mean density used by the pool builder is

$$\bar{n} = \frac{N_e}{L}$$

when `N_e>0`, and the shifted density is

$$\tilde{n}_i = \hat{n}_i - \bar{n}I.$$

Insert the explicit site density:

$$\tilde{n}_i = \left(I - \frac{1}{2}(Z_{p_{i\uparrow}} + Z_{p_{i\downarrow}})\right) - \bar{n}I = (1-\bar{n})I - \frac{1}{2}(Z_{p_{i\uparrow}} + Z_{p_{i\downarrow}}).$$

At half filling $N_e = L$, $\bar{n} = 1$, so

$$\tilde{n}_i = -\frac{1}{2}(Z_{p_{i\uparrow}} + Z_{p_{i\downarrow}}).$$

This is therefore the default algebra used in HH runs when `num_particles` is set to half filling.

10.1.2 Phonon primitives (P_i) and (x_i)

The momentum-like primitive is

$$P_i = i(\hat{b}_i^\dagger - \hat{b}_i).$$

The displacement primitive is

$$x_i = \hat{b}_i + \hat{b}_i^\dagger.$$

In code, these are built from phonon ladder operators on the site's phonon register inside `src/quantum/operator_pools/polaron`

- `p_i(site) = i*(b_i^† - b_i)` gives (P_i),
- `x_i(site)` multiplies `boson_displacement_operator(..., which="x")` so (x_i=b_i+b_i^†).

10.1.3 Local squeeze primitive

The newly added squeeze-like phonon primitive used by the probe families is

$$S_i = i\left((\hat{b}_i^\dagger)^2 - \hat{b}_i^2\right).$$

In code this is built as

$$S_i = i\,\hat{b}_i^\dagger\hat{b}_i^\dagger - i\,\hat{b}_i\hat{b}_i,$$

with intermediate polynomial products cleaned by the same PAOP pruning / real-coefficient rules used for the other generators.

10.1.4 Doublon primitive

The local doublon primitive is the explicit operator from Section 3.3:

$$\hat{d}_i = \frac{1}{4}\Big(I - Z_{p_{i\uparrow}} - Z_{p_{i\downarrow}} + Z_{p_{i\uparrow}}Z_{p_{i\downarrow}}\Big).$$

10.1.5 Even hopping channel

The even hopping channel is

$$K_{ij} = \sum_\sigma \left(\hat{c}_{i\sigma}^\dagger\hat{c}_{j\sigma} + \hat{c}_{j\sigma}^\dagger\hat{c}_{i\sigma}\right).$$

10.1.6 Odd current channel

The odd current channel is

$$J_{ij} = i\sum_\sigma \left(\hat{c}_{i\sigma}^\dagger\hat{c}_{j\sigma} - \hat{c}_{j\sigma}^\dagger\hat{c}_{i\sigma}\right).$$

For one spin channel with mode indices `p_< < p_>`, the implemented JW current primitive is

$$J_{p_< p_>} = \frac{1}{2}\Big(X_{p_>}Z_{p_>-1}\cdots Z_{p_<+1}Y_{p_<} - Y_{p_>}Z_{p_>-1}\cdots Z_{p_<+1}X_{p_<}\Big).$$

The code sums that object over both spins.

10.2 Implemented PAOP channels

The implemented channel families are not abstract labels only; they are explicit algebraic generators.

10.2.1 Conditional displacement

$$\mathcal{O}_{\mathrm{disp},i} = \tilde{n}_i P_i.$$

10.2.2 Legacy doublon dressing

$$\mathcal{O}_{\mathrm{dbl},i} = \tilde{n}_i \hat{d}_i.$$

10.2.3 Dressed hopping drag

$$\mathcal{O}_{\mathrm{hopdrag},ij} = K_{ij}(P_i - P_j).$$

10.2.4 Odd-channel drag

$$\mathcal{O}_{\mathrm{curdrag},ij} = J_{ij}(P_i - P_j).$$

10.2.5 Second-order even channel

$$\mathcal{O}_{\mathrm{hop2},ij} = K_{ij}(P_i - P_j)^2.$$

In the implementation, this family drops terms that are identity on every phonon qubit after the polynomial is formed, so the retained generator keeps explicit phonon support.

10.2.6 Third-order odd current channel

The newly added LF third-order odd channel is

$$\mathcal{O}_{\mathrm{curdrag3},ij} = J_{ij}(P_i - P_j)^3.$$

The concrete builder label is - `paop_curdrag3(i,j)`.

10.2.7 Fourth-order even hopping channel

The newly added LF fourth-order even channel is

$$\mathcal{O}_{\mathrm{hop4},ij} = K_{ij}(P_i - P_j)^4.$$

As with `hop2`, the implementation drops terms that are identity on all phonon qubits after the product is expanded and cleaned.

The concrete builder label is - `paop_hop4(i,j)`.

10.2.8 Bond-conditioned displacement and squeeze channels

The newly added bond-conditioned channels are

$$\mathcal{O}_{\mathrm{bond_disp},ij} = K_{ij}(P_i + P_j),$$

$$\mathcal{O}_{\mathrm{hop_sq},ij} = K_{ij}(S_i + S_j),$$

$$\mathcal{O}_{\mathrm{pair_sq},ij} = i\left(\hat{b}_i^\dagger \hat{b}_j^\dagger - \hat{b}_i \hat{b}_j\right).$$

The concrete builder labels are - `paop_bond_disp(i,j)`, - `paop_hop_sq(i,j)`, - `paop_pair_sq(i,j)`.

10.2.9 Local squeeze channels

The newly added squeeze-family channels are

$$\mathcal{O}_{\mathrm{sq},i} = S_i, \qquad \mathcal{O}_{\mathrm{dens_sq},i} = \tilde{n}_i S_i.$$

The concrete builder labels are - `paop_sq(site=i)`, - `paop_dens_sq(site=i)`.

10.2.10 Extended cloud channels

For cloud radius `R`, the implemented extended cloud channels are

$$\mathcal{O}_{\mathrm{cloud_p},i\to j} = \tilde{n}_i P_j, \qquad \mathcal{O}_{\mathrm{cloud_x},i\to j} = \tilde{n}_i x_j, \qquad \mathcal{O}_{\mathrm{cloud_sq},i\to j} = \tilde{n}_i S_j,$$

with the distance gate `\operatorname{dist}(i,j)\le R`.

The concrete operator labels emitted by the builder are - `paop_cloud_p(site=i->phonon=j)`, - `paop_cloud_x(site=i->phonon=` - `paop_cloud_sq(site=i->phonon=j)`.

For `paop_full`, `paop_lf_full`, and `paop_sq_full`, effective radius is forced to at least 1 (`--paop-r=0` maps to nearest-neighbor cloud support).

10.2.11 Doublon-translation and doublon-squeeze channels

The implemented doublon-conditioned phonon channels are

$$\mathcal{O}_{\mathrm{dbl_p},i\to j} = \hat{d}_i P_j, \qquad \mathcal{O}_{\mathrm{dbl_x},i\to j} = \hat{d}_i x_j, \qquad \mathcal{O}_{\mathrm{dbl_sq},i\to j} = \hat{d}_i S_j.$$

The concrete labels are - `paop_dbl_p(site=i->phonon=j)`, - `paop_dbl_x(site=i->phonon=j)`, - `paop_dbl_sq(site=i->phonon`

10.3 Pool-family map

The current PAOP-family map is:

- `paop` = `paop_std`
- `paop_lf` = `paop_lf_std`
- `paop_min` = `disp`
- `paop_std` = `disp + hopdrag`
- `paop_full` = `disp + doublon + hopdrag + cloud_p + cloud_x`
- `paop_lf_std` = `disp + hopdrag + curdrag`
- `paop_lf2_std` = `disp + hopdrag + curdrag + hop2`
- `paop_lf3_std` = `paop_lf2_std + curdrag3` *(experimental / offline-probe-only)*
- `paop_lf4_std` = `paop_lf3_std + hop4` *(experimental / offline-probe-only)*
- `paop_lf_full` = `disp + hopdrag + curdrag + hop2 + cloud_p + cloud_x + dbl_p + dbl_x`
- `paop_sq_std` = `disp + hopdrag + curdrag + sq + dens_sq` *(experimental / offline-probe-only)*
- `paop_sq_full` = `paop_sq_std + cloud_sq + dbl_sq` *(experimental / offline-probe-only)*
- `paop_bond_disp_std` = `disp + hopdrag + curdrag + bond_disp` *(experimental / offline-probe-only)*
- `paop_hop_sq_std` = `disp + hopdrag + curdrag + hop_sq` *(experimental / offline-probe-only)*
- `paop_pair_sq_std` = `disp + hopdrag + curdrag + pair_sq` *(experimental / offline-probe-only)*

When one of these PAOP families is selected on the HH ADAPT path and `g_{ep}\neq 0`, the live pool builder returns the deduplicated merge

$$\mathcal{P}_{\mathrm{HH,selected}} = \mathcal{P}_{\mathrm{hva}} \cup \mathcal{P}_{\mathrm{hh_termwise_augmented}} \cup \mathcal{P}_{\mathrm{selected_paop_family}}.$$

If `g_{ep}=0`, the implementation returns the selected PAOP family without the HH augmentation merge.

The built-in broad family presets are

$$\mathcal{P}_{\mathrm{uccsd_paop_lf_full}} = \mathcal{P}_{\mathrm{uccsd_lifted}} \cup \mathcal{P}_{\mathrm{paop_lf_full}},$$

$$\mathcal{P}_{\mathrm{full_meta}} = \mathcal{P}_{\mathrm{uccsd_lifted}} \cup \mathcal{P}_{\mathrm{hva}} \cup \mathcal{P}_{\mathrm{hh_termwise_augmented}} \cup \mathcal{P}_{\mathrm{paop_full}} \cup \mathcal{P}_{\mathrm{paop_lf_full}},$$

with the `hh_termwise_augmented` block active when `g_{ep}\neq 0`.

10.3.1 Standalone VLF/SQ macro families

Separate from `polaron_paop.py`, the repo now exposes grouped macro-generator families in `src/quantum/operator_pools/vlf_`

Define the shell-resolved VLF macro generator

$$G_r^{\mathrm{VLF}} = \sum_{\mathrm{dist}(i,j)=r} \tilde{n}_i P_j,$$

and the global squeeze generators

$$G^{\mathrm{SQ}} = \sum_i S_i = \frac{1}{2}\sum_i (x_i P_i + P_i x_i),$$

$$G_{\mathrm{SQ}}^{(n)} = \sum_i \tilde{n}_i S_i.$$

The implemented family map is

- `vlf_only` = `{G_r^{\mathrm{VLF}}}` over the allowed shells,
- `sq_only` = `{G^{\mathrm{SQ}}}`,
- `vlf_sq` = `{G_r^{\mathrm{VLF}}} \cup \{G^{\mathrm{SQ}}\}`,
- `sq_dens_only` = `{G_{\mathrm{SQ}}^{(n)}}`,
- `vlf_sq_dens` = `{G_r^{\mathrm{VLF}}} \cup \{G^{\mathrm{SQ}}, G_{\mathrm{SQ}}^{(n)}\}`.

These are emitted as grouped macro generators such as `- vlf_only:vlf_shell(r=0)`, `- vlf_sq:sq_global`, `- vlf_sq_dens:dens_sq_global`.

Unlike the PAOP families above, the VLF/SQ families are returned directly as macro families and the implementation rejects `--paop-split-paulis` for them.

The implementation then optionally applies

- pruning,
- normalization (`none`, `fro`, `maxcoeff`),
- signature deduplication.

10.4 Legacy fallback selector

The ADAPT signal remains

$$g_m^{(n)} = i\langle \psi^{(n)} | [\hat{H}, A_m] | \psi^{(n)} \rangle,$$

and on the compiled production path this is evaluated as

$$g_m^{(n)} = 2\,\Im\langle H\psi^{(n)} \mid A_m \psi^{(n)} \rangle,$$

with

$$|\psi^{(n)}\rangle = \exp(-i\theta_n A_{m_n}) \cdots \exp(-i\theta_1 A_{m_1}) |\psi_{\mathrm{ref}}\rangle.$$

If staged continuation scoring is disabled, the selector reduces to the legacy rule

$$m_\star = \arg\max_{m \in \mathcal{P}_{\mathrm{avail}}} |g_m^{(n)}|.$$

If repeated operator reuse is allowed, the live code applies the repeat-biased surrogate

$$\tilde{s}_m^{\mathrm{repeat}} = \frac{|g_m^{(n)}|}{1 + \beta\, c_m}, \qquad \beta = 1.5,$$

where `c_m` is the prior selection count of pool element `m`. The chosen legacy operator is then

$$m_\star = \arg\max_{m \in \mathcal{P}_{\mathrm{avail}}} \tilde{s}_m^{\mathrm{repeat}}.$$

10.5 `phase1_v1` as a self-contained selector

10.5.1 Core signal, append position, and refit window

At ADAPT depth `n`, the ordered scaffold state is

$$|\psi^{(n)}\rangle = \exp(-i\theta_n A_{m_n}) \cdots \exp(-i\theta_1 A_{m_1})|\psi_{\mathrm{ref}}\rangle.$$

For a candidate generator `A_m`, the compiled production gradient is

$$g_m^{(n)} = i\langle\psi^{(n)}|[H, A_m]|\psi^{(n)}\rangle = 2\,\Im\langle H\psi^{(n)} \mid A_m\psi^{(n)}\rangle.$$

The `phase1_v1` selector first ranks the available pool by descending gradient magnitude and keeps the cheap candidate universe

$$\mathcal{C}_{64} = \text{top-}\min\{64, |\mathcal{P}_{\mathrm{avail}}|\} \text{ candidates ranked by } |g_m^{(n)}|.$$

The append position is

$$p_{\mathrm{app}} = n.$$

For a tentative insertion position `p`, the active reoptimization window is built from the current parameter vector `\theta` by

1. keeping the newest `w_{\mathrm{eff}}=\min\{w,n\}` indices,
2. then optionally adding the top-`k` older indices ranked by descending `|\theta_j|`, tie-breaking by ascending index.

So the active refit set is

$$W_{\mathrm{refit}}(p) = W_{\mathrm{newest}} \cup W_{\mathrm{top}|\theta|}.$$

10.5.2 Gates, burdens, and feature ingredients

For each candidate-position pair `(m,p)` in the cheap scan, the feature build records

- signed and absolute gradients `g_m`, `|g_m|`,
- lower-confidence gradient `g_{\mathrm{lcb}}`,
- raw tangent surrogate `F_{\mathrm{raw}}`,
- compile-cost proxy,
- grouped-measurement bookkeeping,
- candidate family / stage metadata,
- symmetry metadata,
- predicted refit window.

The stage gate is

$$\Gamma_{\mathrm{stage}}(m) = \begin{cases} 1, & \text{stage} = \text{residual}, \\ 1, & \text{stage} = \text{core/seed and } m \notin \mathcal{P}_{\mathrm{residual}}, \\ 0, & \text{stage} = \text{core/seed and } m \in \mathcal{P}_{\mathrm{residual}}. \end{cases}$$

The hard symmetry gate is

$$\Gamma_{\mathrm{sym}}(m) = \begin{cases} 0, & \text{candidate symmetry spec has } \texttt{hard_guard=true}, \\ 1, & \text{otherwise}. \end{cases}$$

If either gate is closed, the record is not admissible.

The compile-cost proxy used inside the score is

$$D_{\mathrm{proxy}} = n_{\mathrm{new\,pauli}} + n_{\mathrm{rot}} + |p_{\mathrm{app}} - p| + |W_{\mathrm{refit}}(p)|.$$

The grouped-measurement bookkeeping tracks

$$G_{\text{new}}, \qquad S_{\text{new}}, \qquad R_{\text{reuse}},$$

for new groups, new nominal-shot burden, and grouped-reuse miss count.

The lower-confidence gradient is

$$g_{\text{lcb}} = \max\{|g| - z_\alpha \sigma, 0\}.$$

In the current live `phase1_v1` implementation, the feature builder takes

$$\sigma = 0, \qquad F_{\text{raw}} = |g_m^{(n)}|,$$

so the active cheap-path lower-confidence gradient is simply

$$g_{\text{lcb}} = |g_m^{(n)}|.$$

10.5.3 Cheap score

The cheap score used by `phase1_v1` is

$$S_{\text{simple}} = \mathbf{1}(m \in \Omega_{\text{stage}}) \frac{g_{\text{lcb}}^2}{2\lambda_F F_{\text{raw}}} \frac{1}{K_{\text{screen}}},$$

with

$$K_{\text{screen}} = 1 + w_D \bar{D}_{\text{life}} + w_G \bar{G}_{\text{new}} + w_C \bar{C}_{\text{new}} + w_c \bar{c}.$$

Here

- `D_life` is the compiled burden proxy, optionally lifetime-weighted,
- `G_new` is new grouped-measurement burden,
- `C_new` is new nominal shot burden,
- `c` is the same-family streak burden.

The live defaults are

- `lambda_F = 1.0`,
- `lambda_compile = 0.05`,
- `lambda_measure = 0.02`,
- `lambda_leak = 0.0`,
- `z_alpha = 0.0`.

So the live `phase1_v1` cheap screen uses

- `g_{\mathrm{lcb}}=|g|`,
- no active leakage penalty,
- no active additive motif bonus term.

10.5.4 Position probing, trough detection, and effective selector

`phase1_v1` does not force append-only behavior. Outside the residual stage, it probes alternate insertion positions when one of the implemented triggers fires:

- drop plateau,
- `eps_grad` plus flat finite-angle behavior,
- repeated-family flatness.

If probing is enabled, the allowed candidate positions are

$$\mathcal{P}_{\text{probe}} = \{p_{\text{app}}, 0\} \cup W_{\text{refit}}(p_{\text{app}}),$$

truncated to the configured maximum number of probe positions.

The score `S_{\mathrm{simple}}` is then evaluated over

$$m \in \mathcal{C}_{64}, \qquad p \in \mathcal{P}_{\text{probe}}.$$

The trough detector compares the best append score against the best non-append score. A trough is declared when

$$g_{\text{lcb}}^{\text{nonappend}} > 0$$

and either

$$S_{\text{nonappend}} \geq \mu\, S_{\text{append}},$$

or

$$S_{\text{append}} < \tau_{\text{append}} \quad \text{and} \quad S_{\text{nonappend}} \geq \tau_{\text{append}},$$

where `\mu` is the configured probe-margin ratio and `\tau_{\mathrm{append}}` is the append-admit threshold.

So the effective `phase1_v1` selector is:

- build cheap candidate-position records,
- evaluate `S_{\mathrm{simple}}`,
- admit the best surviving record.

10.6 `phase2_v1` as a self-contained selector

10.6.1 Core signal, append position, refit window, and cheap stage

At ADAPT depth `n`, the ordered scaffold state is

$$|\psi^{(n)}\rangle = \exp(-i\theta_n A_{m_n}) \cdots \exp(-i\theta_1 A_{m_1})|\psi_{\text{ref}}\rangle.$$

For a candidate generator `A_m`, the compiled production gradient is

$$g_m^{(n)} = i\langle\psi^{(n)}|[H, A_m]|\psi^{(n)}\rangle = 2\,\Im\langle H\psi^{(n)} \mid A_m\psi^{(n)}\rangle.$$

The cheap candidate universe is

$$\mathcal{C}_{64} = \text{top-}\min\{64, |\mathcal{P}_{\text{avail}}|\} \text{ candidates ranked by } |g_m^{(n)}|.$$

The append position is

$$p_{\text{app}} = n.$$

For a tentative insertion position `p`, the active reoptimization window is

$$W_{\text{refit}}(p) = W_{\text{newest}} \cup W_{\text{top}|\theta|},$$

where `W_{\mathrm{newest}}` keeps the newest `w_{\mathrm{eff}}=\min\{w,n\}` indices and `W_{\mathrm{top}|\theta|}` optionally adds the top-`k` older indices ranked by descending `|\theta_j|`.

The stage gate is

$$\Gamma_{\text{stage}}(m) = \begin{cases} 1, & \text{stage} = \text{residual}, \\ 1, & \text{stage} = \text{core/seed and } m \notin \mathcal{P}_{\text{residual}}, \\ 0, & \text{stage} = \text{core/seed and } m \in \mathcal{P}_{\text{residual}}. \end{cases}$$

The hard symmetry gate is

$$\Gamma_{\text{sym}}(m) = \begin{cases} 0, & \text{candidate symmetry spec has } \texttt{hard_guard=true}, \\ 1, & \text{otherwise}. \end{cases}$$

The compile-cost proxy is

$$D_{\text{proxy}} = n_{\text{new pauli}} + n_{\text{rot}} + |p_{\text{app}} - p| + |W_{\text{refit}}(p)|.$$

The grouped-measurement bookkeeping tracks

$$G_{\text{new}}, \qquad S_{\text{new}}, \qquad R_{\text{reuse}}.$$

The lower-confidence gradient is

$$g_{\text{lcb}} = \max\{|g| - z_\alpha \sigma, 0\},$$

and the cheap-stage burden is

$$K_{\text{screen}} = 1 + w_D \bar{D}_{\text{life}} + w_G \bar{G}_{\text{new}} + w_C \bar{C}_{\text{new}} + w_c \bar{c}.$$

The cheap score is

$$S_{\text{simple}} = \mathbf{1}(m \in \Omega_{\text{stage}}) \frac{g_{\text{lcb}}^2}{2\lambda_F F_{\text{raw}}} \frac{1}{K_{\text{screen}}}.$$

In the current live cheap path for `phase2_v1`, the feature builder takes

$$\sigma = 0, \qquad F_{\text{raw}} = |g_m^{(n)}|,$$

so the active cheap-path lower-confidence gradient is

$$g_{\text{lcb}} = |g_m^{(n)}|.$$

10.6.2 Shortlist rule

After the cheap pass, `phase2_v1` keeps the shortlist

$$N_{\text{short}} = \min\{N,\, N_{\max},\, \lceil f_{\text{short}} N \rceil\},$$

where

- `N` is the number of cheap records,
- `N_{\max}=\texttt{shortlist\_size}`,
- `f_{\mathrm{short}}=\texttt{shortlist\_fraction}`.

The retained shortlist is ranked by

1. descending cheap score,
2. descending `simple_score`,
3. ascending pool index,
4. ascending insertion position.

10.6.3 Reduced-path geometry, novelty, curvature, and trust-region drop

For each shortlisted record, `phase2_v1` rebuilds an exact local derivative and geometry model at the proposed insertion position:

- recompute the gradient on the current state,
- build the exact inherited-window parameter tangents of the ordered scaffold,
- build the exact inherited-window Hessian block,
- build the candidate self-curvature and mixed candidate-window Hessian vector.

Let `W_m` denote the inherited window that would actually be refit if candidate `m` were admitted. Let `t_m` be the horizontal append tangent and `\{t_j\}_{j\in W_m}` be the exact horizontal tangents of the current scaffold parameters in that window. Then

$$F_{\text{raw}} = \langle t_m, t_m \rangle, \qquad (Q_m)_{jk} = \mathrm{Re}\langle t_j, t_k \rangle, \qquad (q_m)_j = \mathrm{Re}\langle t_j, t_m \rangle.$$

The exact local quadratic model over append parameter `\alpha` and inherited-window displacement `\delta\theta_{W_m}` is

$$E(\alpha, \delta\theta_{W_m}) \approx E_0 + g_m \alpha + \frac{1}{2} h_m \alpha^2 + \alpha\, b_m^\top \delta\theta_{W_m} + \frac{1}{2} \delta\theta_{W_m}^\top H_{W_m} \delta\theta_{W_m}.$$

The inherited-window Hessian block is symmetrized and regularized as

$$M_m = \frac{1}{2}(H_{W_m} + H_{W_m}^\top) + \lambda_H I,$$

with ridge growth until the linear solve is numerically stable.

The reduced curvature is

$$\tilde{h}_m = h_m - b_m^\top M_m^{-1} b_m.$$

The reduced-path metric is

$$F_m^{\mathrm{red}} = F_{\mathrm{raw}} - 2q_m^\top M_m^{-1} b_m + b_m^\top M_m^{-1} Q_m M_m^{-1} b_m.$$

The reduced-path overlap is

$$q_m^{\mathrm{red}} = q_m - Q_m M_m^{-1} b_m.$$

The implemented novelty is the Tikhonov-regularized reduced-path novelty

$$\nu_m = \mathrm{clip}_{[0,1]}\left(1 - \frac{(q_m^{\mathrm{red}})^\top (Q_m + \varepsilon_{\mathrm{nov}} I)^{-1} q_m^{\mathrm{red}}}{F_m^{\mathrm{red}}}\right).$$

The trust-region drop is

$$\Delta E_{\mathrm{TR}} = \max_{|\alpha| \le \rho/\sqrt{F_m^{\mathrm{red}}}} \left[g_{\mathrm{lcb}}|\alpha| - \frac{1}{2}\max(\tilde{h}_m, 0)\alpha^2 \right].$$

Equivalently,

$$\Delta E_{\mathrm{TR}} = \begin{cases} \dfrac{g_{\mathrm{lcb}}^2}{2\max(\tilde{h}_m, 0)}, & \max(\tilde{h}_m, 0) > 0 \text{ and } \dfrac{g_{\mathrm{lcb}}}{\max(\tilde{h}_m, 0)} \le \dfrac{\rho}{\sqrt{F_m^{\mathrm{red}}}}, \\ \dfrac{\rho g_{\mathrm{lcb}}}{\sqrt{F_m^{\mathrm{red}}}} - \dfrac{\rho^2 \max(\tilde{h}_m, 0)}{2F_m^{\mathrm{red}}}, & \text{otherwise.} \end{cases}$$

10.6.4 Full rerank score and effective selector

The full rerank burden is

$$K = 1 + w_D \bar{D}_{\mathrm{life}} + w_G \bar{G}_{\mathrm{new}} + w_C \bar{C}_{\mathrm{new}} + w_c \bar{c}.$$

The full rerank score is

$$S_{\mathrm{full}} = \mathbf{1}(m \in \Omega_{\mathrm{stage}})\, \nu_m^{\gamma_N} \frac{\Delta E_{\mathrm{TR}}}{K}.$$

The live defaults are

- `lambda_F = 1.0`,
- `lambda_H = 10^{-6}`,
- `rho = 0.25`,
- `gamma_N = 1.0`,
- `wD = 0.2`,
- `wG = 0.15`,
- `wC = 0.15`,
- `wc = 0.1`,
- `shortlist_fraction = 0.2`,
- `shortlist_size = 12`,
- `batch_target_size = 2`,
- `batch_size_cap = 3`,
- `batch_near_degenerate_ratio = 0.9`.

So the effective `phase2_v1` selector is:

- cheap-rank candidate-position pairs by `S_{\mathrm{simple}}`,

- keep the configured shortlist,
- rerank that shortlist by `S_{\mathrm{full}}`,
- admit the top full-score record.

The code still exposes batch-related configuration knobs and a compatibility-penalty helper, but the current live branch admits a singleton batch only:

- `phase2_selected_records` is set to the top full-score record,
- `greedy_batch_select(...)` is presently a singleton pass-through helper,
- `phase2_last_batch_selected` remains false in the active branch.

10.7 `phase3_v1` as a self-contained selector

10.7.1 Core signal, append position, refit window, and cheap stage

At ADAPT depth `n`, the ordered scaffold state is

$$|\psi^{(n)}\rangle = \exp(-i\theta_n A_{m_n}) \cdots \exp(-i\theta_1 A_{m_1})|\psi_{\mathrm{ref}}\rangle.$$

For a candidate generator `A_m`, the compiled production gradient is

$$g_m^{(n)} = i\langle\psi^{(n)}|[H, A_m]|\psi^{(n)}\rangle = 2\,\Im\langle H\psi^{(n)} \mid A_m\psi^{(n)}\rangle.$$

The cheap candidate universe is

$$\mathcal{C}_{64} = \text{top-}\min\{64, |\mathcal{P}_{\mathrm{avail}}|\} \text{ candidates ranked by } |g_m^{(n)}|.$$

The append position is

$$p_{\mathrm{app}} = n.$$

For a tentative insertion position `p`, the active reoptimization window is

$$W_{\mathrm{refit}}(p) = W_{\mathrm{newest}} \cup W_{\mathrm{top}|\theta|},$$

where `W_{\mathrm{newest}}` keeps the newest `w_{\mathrm{eff}}=\min\{w,n\}` indices and `W_{\mathrm{top}|\theta|}` optionally adds the top-`k` older indices ranked by descending `|\theta_j|`.

The stage gate is

$$\Gamma_{\mathrm{stage}}(m) = \begin{cases} 1, & \text{stage} = \text{residual}, \\ 1, & \text{stage} = \text{core/seed and } m \notin \mathcal{P}_{\mathrm{residual}}, \\ 0, & \text{stage} = \text{core/seed and } m \in \mathcal{P}_{\mathrm{residual}}. \end{cases}$$

The hard symmetry gate is

$$\Gamma_{\mathrm{sym}}(m) = \begin{cases} 0, & \text{candidate symmetry spec has } \texttt{hard_guard=true}, \\ 1, & \text{otherwise}. \end{cases}$$

The compile-cost proxy is

$$D_{\mathrm{proxy}} = n_{\mathrm{new\,pauli}} + n_{\mathrm{rot}} + |p_{\mathrm{app}} - p| + |W_{\mathrm{refit}}(p)|.$$

The grouped-measurement bookkeeping tracks

$$G_{\mathrm{new}}, \qquad S_{\mathrm{new}}, \qquad R_{\mathrm{reuse}}.$$

The lower-confidence gradient is

$$g_{\mathrm{lcb}} = \max\{|g| - z_\alpha\sigma, 0\},$$

and the cheap-stage burden is

$$K_{\text{screen}} = 1 + w_D \bar{D}_{\text{life}} + w_G \bar{G}_{\text{new}} + w_C \bar{C}_{\text{new}} + w_c \bar{c}.$$

The cheap score is

$$S_{\text{simple}} = \mathbf{1}(m \in \Omega_{\text{stage}}) \frac{g_{\text{lcb}}^2}{2\lambda_F F_{\text{raw}}} \frac{1}{K_{\text{screen}}}.$$

In the current live cheap path for `phase3_v1`, the feature builder takes

$$\sigma = 0, \qquad F_{\text{raw}} = |g_m^{(n)}|,$$

so the active cheap-path lower-confidence gradient is

$$g_{\text{lcb}} = |g_m^{(n)}|.$$

10.7.2 Shortlist, reduced-path geometry, novelty, and full rerank score

After the cheap pass, `phase3_v1` keeps the shortlist

$$N_{\text{short}} = \min\{N,\, N_{\max},\, \lceil f_{\text{short}} N \rceil\},$$

where

- `N` is the number of cheap records,
- `N_{\max}=\texttt{shortlist\_size}`,
- `f_{\mathrm{short}}=\texttt{shortlist\_fraction}`.

For each shortlisted record, `phase3_v1` rebuilds an exact local derivative and geometry model at the proposed insertion position.

Let `W_m` denote the inherited window that would actually be refit if candidate `m` were admitted. Let `t_m` be the horizontal append tangent and `\{t_j\}_{j\in W_m}` the exact horizontal tangents of the current scaffold parameters in that window. Then

$$F_{\text{raw}} = \langle t_m, t_m \rangle, \qquad (Q_m)_{jk} = \mathrm{Re}\langle t_j, t_k \rangle, \qquad (q_m)_j = \mathrm{Re}\langle t_j, t_m \rangle.$$

The local quadratic model is

$$E(\alpha, \delta\theta_{W_m}) \approx E_0 + g_m \alpha + \frac{1}{2} h_m \alpha^2 + \alpha\, b_m^\top \delta\theta_{W_m} + \frac{1}{2} \delta\theta_{W_m}^\top H_{W_m} \delta\theta_{W_m}.$$

The positive-definite inherited-window model is

$$M_m = \frac{1}{2}(H_{W_m} + H_{W_m}^\top) + \lambda_H I,$$

with ridge growth until the solve is stable.

The reduced curvature is

$$\tilde{h}_m = h_m - b_m^\top M_m^{-1} b_m.$$

The reduced-path metric is

$$F_m^{\text{red}} = F_{\text{raw}} - 2 q_m^\top M_m^{-1} b_m + b_m^\top M_m^{-1} Q_m M_m^{-1} b_m.$$

The reduced-path overlap is

$$q_m^{\text{red}} = q_m - Q_m M_m^{-1} b_m.$$

The implemented novelty is

$$\nu_m = \mathrm{clip}_{[0,1]} \left(1 - \frac{(q_m^{\text{red}})^\top (Q_m + \varepsilon_{\text{nov}} I)^{-1} q_m^{\text{red}}}{F_m^{\text{red}}} \right).$$

The trust-region drop is

$$\Delta E_{\mathrm{TR}} = \max_{|\alpha| \le \rho/\sqrt{F_m^{\mathrm{red}}}} \left[g_{\mathrm{lcb}}|\alpha| - \frac{1}{2}\max(\tilde{h}_m, 0)\alpha^2 \right].$$

Equivalently,

$$\Delta E_{\mathrm{TR}} = \begin{cases} \dfrac{g_{\mathrm{lcb}}^2}{2\max(\tilde{h}_m, 0)}, & \max(\tilde{h}_m, 0) > 0 \text{ and } \dfrac{g_{\mathrm{lcb}}}{\max(\tilde{h}_m, 0)} \le \dfrac{\rho}{\sqrt{F_m^{\mathrm{red}}}}, \\ \dfrac{\rho g_{\mathrm{lcb}}}{\sqrt{F_m^{\mathrm{red}}}} - \dfrac{\rho^2 \max(\tilde{h}_m, 0)}{2F_m^{\mathrm{red}}}, & \text{otherwise.} \end{cases}$$

The full rerank burden is

$$K = 1 + w_D \bar{D}_{\mathrm{life}} + w_G \bar{G}_{\mathrm{new}} + w_C \bar{C}_{\mathrm{new}} + w_c \bar{c}.$$

The full rerank score is

$$S_{\mathrm{full}} = \mathbf{1}(m \in \Omega_{\mathrm{stage}})\, \nu_m^{\gamma_N} \frac{\Delta E_{\mathrm{TR}}}{K}.$$

10.7.3 Additional runtime surfaces and effective selector

The live `phase3_v1` surface also carries the following runtime surfaces:

1. **Motif seeding**. If `phase3_motif_source_json` is supplied, the code loads a motif library and may insert transferable tiled generators before the main ADAPT loop.
2. **Runtime split on shortlisted macro generators**. If

   $$\texttt{phase3_runtime_split_mode} = \texttt{shortlist_pauli_children_v1},$$

   and a shortlisted candidate is a macro generator, the code builds child generators

   $$\mathcal{C}(m) = \{m_1, \dots, m_K\},$$

   evaluates the phase-3 full rerank score over those children, and may replace the parent by the best child.
3. **Lifetime-weighted burden**. If

   $$\texttt{phase3_lifetime_cost_mode} = \texttt{phase3_v1},$$

   the depth burden is weighted by

   $$R_{\mathrm{rem}} = \max\{1, \texttt{max_depth} - \texttt{current_depth} + 1\}.$$

4. **Symmetry metadata / verification hooks**. The exposed mode surface is

   $$\texttt{phase3_symmetry_mitigation_mode} \in \{\texttt{off}, \texttt{verify_only}, \texttt{postselect_diag_v1}, \texttt{projector_renorm_v1}\}.$$

5. **Rescue diagnostics**. If `phase3_enable_rescue=true`, the code may record rescue attempts and outcomes in `rescue_history`.

The live defaults are

- `lambda_F = 1.0`,
- `lambda_H = 10^{-6}`,
- `rho = 0.25`,
- `gamma_N = 1.0`,
- `wD = 0.2`,
- `wG = 0.15`,
- `wC = 0.15`,
- `wc = 0.1`,
- `shortlist_fraction = 0.2`,

- `shortlist_size = 12`,
- singleton batch admission in the active branch.

So the effective `phase3_v1` selector is:

- cheap-rank candidate-position pairs by `S_{\mathrm{simple}}`,
- keep the configured shortlist,
- rerank the shortlist by `S_{\mathrm{full}}`,
- optionally runtime-split shortlisted macros,
- admit the top surviving full-score record,
- carry motif/runtime-split/rescue/symmetry/lifetime metadata as part of the live phase-3 surface.

10.8 Current HH pool and replay-family surfaces

The active HH ADAPT surface in `pipelines/hardcoded/adapt_pipeline.py` supports

- `hva`,
- `full_meta`,
- `uccsd_paop_lf_full`,
- `uccsd_otimes_paop_lf_std`, `uccsd_otimes_paop_lf2_std`, `uccsd_otimes_paop_bond_disp_std`,
- `uccsd_otimes_paop_lf_std_seq2p`, `uccsd_otimes_paop_lf2_std_seq2p`, `uccsd_otimes_paop_bond_disp_std_seq2p`,
- `paop`, `paop_min`, `paop_std`, `paop_full`,
- `paop_lf`, `paop_lf_std`, `paop_lf2_std`, `paop_lf3_std`, `paop_lf4_std`, `paop_lf_full`,
- `paop_sq_std`, `paop_sq_full`,
- `paop_bond_disp_std`, `paop_hop_sq_std`, `paop_pair_sq_std`,
- `vlf_only`, `sq_only`, `vlf_sq`, `sq_dens_only`, `vlf_sq_dens`,
- `full_hamiltonian`.

The replay-side family resolver in `hh_vqe_from_adapt_family.py` additionally accepts the compatibility aliases

- `pool_a`,
- `pool_b`,

which are replay-only legacy mappings rather than direct `adapt_pipeline.py` pool keys.

For the new HH product-family surfaces, the canonical generator is

$$G_{a\mu} = F_a M_\mu,$$

where

- F_a is one lifted fermionic UCCSD excitation factor,
- M_μ is one boson-only phonon motif constructed directly from the phonon primitives used by the PAOP families,
- the allowed v1 motif families are `paop_lf_std`, `paop_lf2_std`, and `paop_bond_disp_std`,
- admissible pairs are locality-filtered against the excitation support before canonicalization and deduplication.

So the product-family pool is

$$\mathcal{P}_{\mathrm{uccsd}\otimes\mathrm{motif}} = \left\{ F_a M_\mu \ : \ (a,\mu) \in \mathcal{C}_{\mathrm{local}} \right\},$$

with $\mathcal{C}_{\mathrm{local}}$ the code-level compatibility relation induced by excitation support and motif support.

This is distinct from the older additive union

$$\mathcal{P}_{\mathrm{uccsd_paop_lf_full}} = \mathcal{P}_{\mathrm{uccsd_lifted}} \cup \mathcal{P}_{\mathrm{paop_lf_full}},$$

which remains unchanged.

For the `_seq2p` variants, one logical pair (F_a, M_μ) is exposed with two parameters via the ordered factorization

$$U_{a\mu}(\alpha,\beta) = e^{-i\alpha F_a} e^{-i\beta M_\mu},$$

rather than the single-parameter product exponential

$$U_{a\mu}(\theta) = e^{-i\theta(F_a M_\mu)}.$$

These logical-pair variants are additive opt-in surfaces and do not change the staged `phase3_v1` default core.

For staged HH continuation modes `phase1_v1`, `phase2_v1`, and `phase3_v1`, the code enforces

- no direct `full_meta` request as the depth-0 core pool,
- default narrow core pool `paop_lf_std`,
- residual pool `full_meta`.

So the staged pool is

$$\mathcal{P}_{\text{staged}} = \mathcal{P}_{\text{core}} \cup \mathcal{P}_{\text{residual}},$$

with

$$\mathcal{P}_{\text{core}} = \mathcal{P}_{\text{paop_lf_std}}, \qquad \mathcal{P}_{\text{residual}} = \mathcal{P}_{\text{full_meta}} \backslash \mathcal{P}_{\text{core}}.$$

10.9 Stage controller

The implemented stage controller in `hh_continuation_stage_control.py` uses the stage chain

$$\texttt{seed} \to \texttt{core} \to \texttt{residual}.$$

The `seed` stage is entered first in staged HH continuation and then resolves immediately:

- `seed -> core` after optional motif seeding completes,
- if no seed block is inserted, the code records `seed_skipped_or_empty` and still moves to `core`,
- `core -> residual` when the drop plateau patience is hit **and** no trough is detected,
- `residual` remains `residual` while open.

Position probing is enabled when one of the implemented triggers fires:

- drop plateau,
- `eps_grad` + finite-angle flatness,
- repeated-family flatness.

The live controller defaults are

- `plateau_patience = 2`,
- `probe_margin_ratio = 1.0`,
- `max_probe_positions = 6`,
- `append_admit_threshold = 0.05`,
- `family_repeat_patience = 2`.

Implemented surfaces:

- `src/quantum/operator_pools/polaron_paop.py`
- `pipelines/hardcoded/adapt_pipeline.py`
- `pipelines/hardcoded/hh_continuation_stage_control.py`
- `pipelines/hardcoded/hh_continuation_scoring.py`

11. SPSA and Optimizer Semantics

11.1 SPSA schedules

The implemented SPSA schedules are

$$c_k = \frac{c}{(k+1)^\gamma}, \qquad a_k = \frac{a}{(A+k+1)^\alpha}.$$

11.2 Two-point stochastic gradient

At iteration `k`, with random Rademacher vector `\Delta_k`, the objective is sampled at

$$y_+ = f(x_k + c_k \Delta_k), \qquad y_- = f(x_k - c_k \Delta_k),$$

and the implemented gradient estimate is

$$\hat{g}_k = \frac{y_+ - y_-}{2c_k}\,\Delta_k.$$

11.3 Update and projection

The parameter update is

$$x_{k+1} = x_k - a_k \hat{g}_k.$$

If clipping projection is enabled, then clipping is applied

1. before evaluating `x_+` and `x_-`,
2. after the parameter update.

11.4 Repeat aggregation

If `eval_repeats > 1`, then the same objective point is evaluated multiple times and aggregated by

$$\text{mean} \quad \text{or} \quad \text{median}\,.$$

11.5 Return policy

The implemented return policy is

- if `avg_last > 0`, return the Polyak-style average of the last `avg_last` iterates and evaluate it once more;
- if `avg_last = 0`, return the best observed sampled point among the evaluated `x_+` and `x_-` points.

11.6 Surface note

`pipelines/hardcoded/adapt_pipeline.py` currently defaults the HH ADAPT inner optimizer to `SPSA`, while non-ADAPT hardcoded VQE surfaces may also use deterministic optimizers such as `COBYLA` or `SLSQP`.

Implemented surfaces:

- `src/quantum/spsa_optimizer.py`
- `pipelines/hardcoded/adapt_pipeline.py`
- `pipelines/hardcoded/hubbard_pipeline.py`

12. Drive, Exact Propagation, and Propagator Semantics

12.1 Static drive labels and time-dependent coefficients

The implemented density-drive template precomputes static `Z` labels for each `(site, spin)` pair:

$$(i, \sigma) \mapsto Z_{p(i,\sigma)}.$$

Only the coefficients are time dependent.

So the drive term is always of the form

$$\hat{H}_{\text{drive}}(t) = \sum_{\lambda \in \mathcal{L}_{\text{drive}}} c_\lambda(t) P_\lambda,$$

with a fixed label set `\mathcal L_{\mathrm{drive}}` and time-varying coefficients `c_{\lambda}(t)`.

12.2 Reference coefficient map

For the density-drive helper,

$$\Delta c[Z_{p(i,\sigma)}](t) = -\frac{1}{2} v_i(t),$$

and optionally

$$\Delta c[I](t) = \sum_i v_i(t).$$

This is the direct runtime version of the substituted drive Hamiltonian from Section 6.3.

12.3 Propagator surfaces

The non-ADAPT hardcoded pipeline exposes propagators including

- `suzuki2`,
- `piecewise_exact`,
- `cfqm4`,
- `cfqm6`.

For CFQM surfaces, the repository policy is explicit:

- CFQM ignores midpoint/left/right `drive-time-sampling`,
- CFQM uses its own fixed scheme nodes `c_j`.

So if the macro-step grid is `t_n`, a CFQM stage samples at

$$t_n + c_j \Delta t,$$

not at left, midpoint, or right rule points chosen by a legacy sampler flag.

Code anchor: `pipelines/hardcoded/hubbard_pipeline.py`

12.4 Reference propagator versus reported trajectory propagator

The hardcoded pipeline now uses an explicit split between

1. the **reported trajectory propagator** selected by `--propagator`, and
2. the **reference / exact branch propagator** used for fidelity and exact-observable comparison.

Write

$$N_{\text{macro}} = \texttt{trotter_steps}, \qquad M_{\text{ref}} = \max\{1, \texttt{exact_steps_multiplier}\}, \qquad N_{\text{ref}} = M_{\text{ref}} N_{\text{macro}}.$$

12.4.1 Static reference branch

If the drive is disabled, the reference branch is the exact eigendecomposition evolution

$$\hat{H}_{\text{static}} = V \Lambda V^\dagger, \qquad |\psi_{\text{ref}}(t)\rangle = V e^{-i\Lambda t} V^\dagger |\psi(0)\rangle.$$

12.4.2 Drive-enabled reference branch

If the drive is enabled, the reference branch is no longer built from one static eigendecomposition. The code switches to a piecewise-constant exact reference,

$$|\psi_{\text{ref}}(t)\rangle = \hat{U}^{(\text{ref})}_{\text{piecewise}}(t) |\psi(0)\rangle,$$

with

$$\hat{U}^{(\text{ref})}_{\text{piecewise}}(t) \approx \prod_{r=0}^{N_{\text{ref}}-1} \exp\left[-i\, \Delta t_{\text{ref}}\, \hat{H}(t_r^\star)\right], \qquad \Delta t_{\text{ref}} = \frac{t}{N_{\text{ref}}},$$

where `t_r^*` follows the selected time-sampling rule for the piecewise reference path.

So the implemented refinement law is explicit:

$$N_{\text{ref}} = \texttt{exact_steps_multiplier} \times \texttt{trotter_steps}.$$

This multiplier changes the reference path only. It does **not** change the macro-step count of `cfqm4` or `cfqm6`.

Code anchor: `pipelines/hardcoded/hubbard_pipeline.py`

12.5 CFQM macro-step mathematics

For CFQM propagation the hardcoded pipeline loads a scheme

$$\{(a_m, c_m)\}_{m=1}^{M}$$

from `get_cfqm_scheme(...)` and validates it with `validate_scheme(...)`.

For one macro-step of size `\Delta t`, the implemented CFQM mathematical surface is

$$\hat{U}_{\mathrm{CFQM}}(\Delta t; t_n) \approx \prod_{m=1}^{M} \exp\left[-i\, a_m \Delta t\, \hat{H}(t_n + c_m \Delta t)\right].$$

The exposed scheme ids are

$$\texttt{cfqm4} \equiv \texttt{CF4:2}, \qquad \texttt{cfqm6} \equiv \texttt{CF6:5Opt}.$$

The stage exponential backend is then chosen from

$$\texttt{cfqm_stage_exp} \in \{\texttt{expm_multiply_sparse},\ \texttt{dense_expm},\ \texttt{pauli_suzuki2}\}.$$

The code also states two runtime semantics directly:

- midpoint/left/right sampling is ignored on CFQM paths because the sample nodes are the fixed `c_m`;
- `pauli_suzuki2` changes the stage exponential implementation and therefore collapses the overall method to a second-order inner product-formula realization.

Code anchors:

- `src/quantum/time_propagation/cfqm_schemes.py`
- `src/quantum/time_propagation/cfqm_propagator.py`
- `pipelines/hardcoded/hubbard_pipeline.py`

12.6 Filtered exact manifold and subspace fidelity

The hardcoded pipeline does not compare a propagated trial state only against a single exact vector. It constructs a filtered exact manifold at `t=0` by selecting every exact state in the target sector whose energy satisfies

$$E \le E_0 + \varepsilon_{\mathrm{subspace}}, \qquad \varepsilon_{\mathrm{subspace}} = \texttt{fidelity_subspace_energy_tol}.$$

If the resulting orthonormal basis at `t=0` is

$$V_0 = [|v_1(0)\rangle, \dots, |v_r(0)\rangle],$$

then each basis vector is propagated by the same reference propagator used for the exact branch, producing

$$V(t) = [|v_1(t)\rangle, \dots, |v_r(t)\rangle].$$

After re-orthonormalization the time-dependent projector is

$$\hat{P}_{\mathrm{gs,filtered}}(t) = V(t)V(t)^{\dagger}.$$

The reported projected fidelities are therefore

$$F_{\mathrm{legacy}}(t) = \langle \psi_{\mathrm{legacy,trot}}(t)|\hat{P}_{\mathrm{gs,filtered}}(t)|\psi_{\mathrm{legacy,trot}}(t)\rangle,$$
$$F_{\mathrm{paop}}(t) = \langle \psi_{\mathrm{paop,trot}}(t)|\hat{P}_{\mathrm{gs,filtered}}(t)|\psi_{\mathrm{paop,trot}}(t)\rangle,$$
$$F_{\mathrm{hva}}(t) = \langle \psi_{\mathrm{hva,trot}}(t)|\hat{P}_{\mathrm{gs,filtered}}(t)|\psi_{\mathrm{hva,trot}}(t)\rangle.$$

The compatibility key `fidelity` follows the currently selected legacy branch, while the branch-resolved keys

- `fidelity_paop_trotter`,
- `fidelity_hva_trotter`

remain explicit in the payload.

Code anchor: `pipelines/hardcoded/hubbard_pipeline.py`

12.7 Branch-resolved observable contracts

The hardcoded pipeline now propagates several explicit branch states instead of only one ansatz branch:

1. filtered exact-sector ground-state reference,
2. exact propagation from the PAOP / ADAPT handoff state,
3. Trotter or CFQM propagation from the PAOP / ADAPT handoff state,
4. exact propagation from the hardcoded VQE branch,
5. Trotter or CFQM propagation from the hardcoded VQE branch.

Write these as

$$|\psi_{\mathrm{gs,exact}}(t)\rangle,\ |\psi_{\mathrm{paop,exact}}(t)\rangle,\ |\psi_{\mathrm{paop,prop}}(t)\rangle,\ |\psi_{\mathrm{hva,exact}}(t)\rangle,\ |\psi_{\mathrm{hva,prop}}(t)\rangle.$$

The static-energy observable on any branch `b` is

$$E^{(b)}_{\mathrm{static}}(t) = \langle\psi_b(t)|\hat{H}_{\mathrm{static}}|\psi_b(t)\rangle.$$

If the drive is enabled, the total instantaneous energy on branch `b` is

$$E^{(b)}_{\mathrm{total}}(t) = \left\langle\psi_b(t)\left|\hat{H}_{\mathrm{static}} + \hat{H}_{\mathrm{drive}}(t_{\mathrm{drive}})\right|\psi_b(t)\right\rangle, \qquad t_{\mathrm{drive}} = \texttt{drive_t0} + t.$$

The site-resolved densities and doublon observables are evaluated branchwise as

$$n^{(b)}_{i\uparrow}(t) = \langle\psi_b(t)|\hat{n}_{i\uparrow}|\psi_b(t)\rangle, \qquad n^{(b)}_{i\downarrow}(t) = \langle\psi_b(t)|\hat{n}_{i\downarrow}|\psi_b(t)\rangle,$$

$$n^{(b)}_{i}(t) = n^{(b)}_{i\uparrow}(t) + n^{(b)}_{i\downarrow}(t), \qquad D^{(b)}(t) = \sum_i \langle\psi_b(t)|\hat{n}_{i\uparrow}\hat{n}_{i\downarrow}|\psi_b(t)\rangle.$$

When the drive is disabled, the code path reduces to

$$E^{(b)}_{\mathrm{total}}(t) = E^{(b)}_{\mathrm{static}}(t).$$

Code anchor: `pipelines/hardcoded/hubbard_pipeline.py`

12.8 `adapt_json` import and handoff contract

The hardcoded pipeline exposes an explicit import surface

$$\texttt{initial_state_source} = \texttt{adapt_json},$$

with the required input path

$$\texttt{adapt_input_json}.$$

The imported statevector is the amplitude payload written by the handoff bundle path, so the mathematical initial condition is

$$|\psi_{\mathrm{paop}}(0)\rangle = |\psi_{\mathrm{imported\ adapt}}(0)\rangle.$$

The hardcoded pipeline also performs a physics-setting comparison between the current run arguments and the imported JSON metadata. If the mismatch set is

$$\mathcal{M} = \{\text{field} \mid \text{current setting} \neq \text{imported setting}\},$$

then:

- if `adapt_strict_match = true` and `\mathcal M\neq \varnothing`, execution stops with an error;
- if `adapt_strict_match = false`, the mismatches are retained as provenance and the import continues.

The imported-handoff payload therefore has two simultaneous meanings:

1. it defines the propagated PAOP branch initial state,
2. it defines an auditable metadata match or mismatch contract through fields such as

- metadata_match_passed,
- metadata_mismatches,
- pool_type,
- ansatz_depth,
- energy,
- abs_delta_e.

Code anchors:

- pipelines/hardcoded/hubbard_pipeline.py
- pipelines/hardcoded/handoff_state_bundle.py

12.9 Exposed phase-3 continuation surfaces

The hardcoded pipeline now exposes several continuation-era control surfaces even when this manuscript does not upgrade them into stronger mathematical claims than the code presently supports.

The exposed continuation-mode surface is

$$\texttt{adapt_continuation_mode} \in \{\texttt{legacy}, \texttt{phase1_v1}, \texttt{phase2_v1}, \texttt{phase3_v1}\}.$$

The exposed symmetry-mitigation mode names are

$$\texttt{phase3_symmetry_mitigation_mode} \in \{\texttt{off}, \texttt{verify_only}, \texttt{postselect_diag_v1}, \texttt{projector_renorm_v1}\}.$$

The exposed runtime split surface is

$$\texttt{phase3_runtime_split_mode} \in \{\texttt{off}, \texttt{shortlist_pauli_children_v1}\}.$$

Additional phase-3-adjacent control surfaces include

- phase3_motif_source_json,
- phase3_enable_rescue,
- phase3_lifetime_cost_mode.

This manuscript therefore treats those names as **implemented runtime surfaces**. It does **not** promote every broader continuation-theory claim from the design notes into a present-tense statement unless a code anchor is present in the main implementation.

Implemented surfaces:

- src/quantum/drives_time_potential.py
- src/quantum/time_propagation/cfqm_propagator.py
- src/quantum/time_propagation/cfqm_schemes.py
- pipelines/hardcoded/hubbard_pipeline.py

13. Implemented Continuation, Handoff, and Replay Contract

13.1 Handoff state bundle

The canonical non-interactive handoff surface is pipelines/hardcoded/handoff_state_bundle.py.

The written payload contains

- the HH settings manifest,
- the ADAPT/VQE energy summary,
- the normalized statevector amplitudes in q_(N_q-1)...q_0 order,
- the exact sector energy,
- the optional continuation block.

The amplitude dictionary is

$$\{\texttt{bitstring}_{q_{N_q-1}\cdots q_0} \mapsto (\Re a, \Im a)\},$$

with all amplitudes below the chosen cutoff omitted.

13.2 Continuation block

The continuation payload is no longer hypothetical. The implemented continuation block may contain

- `mode`,
- `scaffold`,
- `optimizer_memory`,
- `selected_generator_metadata`,
- `generator_split_events`,
- `runtime_split_summary`,
- `motif_library`,
- `motif_usage`,
- `symmetry_mitigation`,
- `rescue_history`,
- `replay_contract`,
- optionally `replay_contract_hint` as a compatibility hint written by bundle tooling.

A schematic shape is

```yaml
continuation:
  mode: phase1_v1 | phase2_v1 | phase3_v1 | legacy
  scaffold: {...}
  optimizer_memory: {...}
  selected_generator_metadata: [...]
  generator_split_events: [...]
  runtime_split_summary: {...}
  motif_library: {...}
  motif_usage: {...}
  symmetry_mitigation: {...}
  rescue_history: [...]
  replay_contract: {...}
  replay_contract_hint: {...}   # compatibility-only hint when present
```

This is the implemented successor to the older continuation discussion.

13.3 Replay contract

The replay-side consumer is the continuation family/replay stack under

- `pipelines/hardcoded/hh_vqe_from_adapt_family.py`,
- `pipelines/hardcoded/hh_continuation_generators.py`,
- `pipelines/hardcoded/hh_continuation_motifs.py`,
- `pipelines/hardcoded/hh_continuation_symmetry.py`.

The mathematical meaning of the replay payload is straightforward:

- the statevector amplitudes define the initial state,
- the continuation block defines how the selected generators were staged, split, scored, reused, and annotated,
- the replay consumer reconstructs a compatible continuation trajectory from those stored objects.

Modern replay treats

`continuation.replay_contract`

as the authoritative contract when present. The implemented v2 shape is

```yaml
continuation:
  replay_contract:
    contract_version: 2
    continuation_mode: legacy | phase1_v1 | phase2_v1 | phase3_v1
    generator_family:
      requested: match_adapt | <canonical_hh_family>
      resolved: <canonical_hh_family>
```

```
      resolution_source: <string>
      fallback_family: <canonical_hh_family> | null
      fallback_used: true | false
    seed_policy_requested: auto | scaffold_plus_zero | residual_only | tile_adapt
    seed_policy_resolved: scaffold_plus_zero | residual_only | tile_adapt
    handoff_state_kind: prepared_state | reference_state
    provenance_source: contract | explicit | inferred_source | ambiguous
    adapt_depth: <int>
    reps: <int>
    derived_num_parameters_formula: adapt_depth * reps
    derived_num_parameters: <int>
    replay_block_source: adapt_vqe.operators
    seed_source: adapt_vqe.optimal_point
```

The live auto-resolution rule is

- `auto -> residual_only` for `handoff_state_kind = prepared_state`,
- `auto -> scaffold_plus_zero` for `handoff_state_kind = reference_state`.

So `replay_contract` now carries the authoritative family resolution, seed-policy resolution, handoff-state semantics, and replay provenance. When `replay_contract` is present, any remaining `replay_contract_hint` should be read as compatibility metadata rather than the primary contract.

13.4 Symmetry and phase-3 payload note

The raw staged ADAPT CLI already exposes phase-3 symmetry, runtime-split, motif, and rescue surfaces. This manuscript therefore states them as implemented payload/data-contract surfaces without overstating raw staged-ADAPT enforcement beyond what the current code actually does.

13.5 Optional staged seed-refine insertion and provenance

Code anchors:

- `pipelines/hardcoded/hh_staged_workflow.py`
- `pipelines/hardcoded/hh_vqe_from_adapt_family.py`
- `pipelines/hardcoded/handoff_state_bundle.py`

The default staged HH chain remains

$$|\psi_{\mathrm{HF}}\rangle \xrightarrow{U_{\mathrm{warm}}(\theta)} |\psi_{\mathrm{warm}}\rangle \xrightarrow{\mathrm{ADAPT}_{\mathrm{phase3_v1}}} |\psi_{\mathrm{ADAPT}}\rangle \xrightarrow{\mathrm{replay}_{\mathrm{match_adapt}}} |\psi_{\mathrm{final}}\rangle,$$

with U_{warm} built from `hh_hva_ptw` by default and `hh_hva` kept as an override-only alternative.

When `--seed-refine-family` is enabled, the workflow inserts one explicit-family conventional VQE stage:

$$|\psi_{\mathrm{HF}}\rangle \xrightarrow{U_{\mathrm{warm}}(\theta)} |\psi_{\mathrm{warm}}\rangle \xrightarrow{U_{\mathrm{family}}(\vartheta)} |\psi_{\mathrm{refined}}\rangle \xrightarrow{\mathrm{ADAPT}_{\mathrm{phase3_v1}}} |\psi_{\mathrm{ADAPT}}\rangle \xrightarrow{\mathrm{replay}_{\mathrm{match_adapt}}} |\psi_{\mathrm{final}}\rangle.$$

The family stage is explicit-only:

- it materializes the requested family directly,
- it does not use `match_adapt`,
- it does not auto-fallback to `full_meta`,
- failure aborts before ADAPT rather than silently skipping ahead.

When a refine stage is present, the handoff bundle may include additive provenance

```
seed_provenance:
  warm_ansatz: hh_hva_ptw | hh_hva
  refine_family: <explicit_family>
  refine_family_kind: explicit_family | <future-kind>
```

```
refine_paop_motif_families: [<motif_family>, ...]
refine_reps: <int>
```

Missing `seed_provenance` remains a valid legacy state and means "no refine stage". Replay family resolution ignores this block; it remains provenance only.

14. Cross-Check Suite and Exact-Benchmark Contracts

14.1 Trial matrix by problem family

Code anchor: `pipelines/exact_bench/cross_check_suite.py`

The cross-check suite implements an exact-benchmark matrix over ansatz families and ADAPT modes.

For pure Hubbard the implemented trial set is

$$\mathcal{T}_{\text{Hub}} = \{\text{HVA-Layerwise}, \text{UCCSD-Layerwise}, \text{ADAPT(UCCSD)}, \text{ADAPT(full_H)}\}.$$

For Hubbard-Holstein the implemented trial set is

$$\mathcal{T}_{\text{HH}} = \{\text{HH-Termwise}, \text{HH-Layerwise}, \text{ADAPT(full_H)}\}.$$

This remains the default HH matrix. When the opt-in seed surface is enabled via `--hh-seed-refine-surface`, the suite additively appends the conventional seed comparison arms

$$\mathcal{T}_{\text{HH}}^{\text{seed}} = \mathcal{T}_{\text{HH}} \cup \{\text{HH-PhysicalTermwise}, \text{HH-PhysicalTermwise+}\texttt{uccsd_paop_lf_full}, \text{HH-PhysicalTermwise+}\texttt{uccsd_otimes_pac}$$

On the smallest-system comparison only, the suite additionally includes the reduced arm

$$\text{HH-Layerwise} + \texttt{uccsd_otimes_paop_lf_std}.$$

Each trial uses the same Hamiltonian instance for the chosen problem and then compares its variational energy against the same sector-filtered exact reference.

14.2 Auto-scaled parameter resolution

Code anchor: `pipelines/exact_bench/cross_check_suite.py`

The suite resolves its runtime parameters by an explicit override-or-table rule. If `x` is a configurable parameter such as

- `vqe_reps`,
- `vqe_restarts`,
- `vqe_maxiter`,
- `vqe_method`,
- `trotter_steps`,
- `num_times`,
- `t_final`,

then the resolved value is

$$x_{\text{resolved}} = \begin{cases} x_{\text{CLI}}, & x_{\text{CLI}} \neq \emptyset, \\ x_{\text{table}}, & x_{\text{CLI}} = \emptyset. \end{cases}$$

For HH runs the suite also enforces the method contract

$$\texttt{vqe_method} = \texttt{SPSA},$$

so HH cross-checks do not silently drop onto legacy deterministic optimizer settings.

The three table sources are separated in code:

- Hubbard trajectory / VQE defaults from `_get_hubbard_params(L)`,
- HH enrichment from `_get_hh_params(L,n_{\mathrm{ph,max}})`,
- ADAPT defaults from `_get_adapt_params(L)`.

14.3 Exact target and shared reference-state construction

Code anchor: `pipelines/exact_bench/cross_check_suite.py`

The cross-check suite constructs one Hamiltonian matrix

$$\hat{H} \longrightarrow H_{\text{mat}},$$

then computes the exact sector-filtered target

$$E_{\text{exact,filtered}} = \min_{\psi \in \mathcal{S}_{\text{sector}}} \langle \psi | \hat{H} | \psi \rangle.$$

For HH, the common reference state is

$$|\psi_{\text{ref}}^{\text{HH}}\rangle = |\text{vac}_{\text{ph}}\rangle \otimes |\Phi_{\text{HF}}\rangle.$$

For pure Hubbard, the common reference state is the HF basis state

$$|\psi_{\text{ref}}^{\text{Hub}}\rangle = |\Phi_{\text{HF}}\rangle.$$

So every variational trial begins from the same physical reference within a given problem instance, and only the ansatz family or ADAPT pool changes.

14.4 Per-trial energy and trajectory semantics

Code anchor: `pipelines/exact_bench/cross_check_suite.py`

If trial `\tau` returns variational state `|\psi_{\tau}\rangle`, then its benchmark energy gap is

$$\Delta E_\tau = E_\tau - E_{\text{exact,filtered}}, \qquad |\Delta E_\tau| = \texttt{delta_E_abs}.$$

After VQE or ADAPT state preparation, every trial is propagated on the same common grid:

$$t_n \in \text{linspace}(0, t_{\text{final}}, N_t), \qquad N_t = \texttt{num_times}.$$

The propagated trajectory of each trial then stores the same observable families:

- projected fidelity,
- exact and Trotter energy,
- site-0 spin occupations,
- total doublon.

So the trajectory comparison is controlled: the ansatz family changes, but the Hamiltonian, reference target, and propagation grid do not.

14.5 JSON and PDF artifact contracts

Code anchor: `pipelines/exact_bench/cross_check_suite.py`

The machine-readable payload begins with

$$\{\texttt{cross_check_suite} : \texttt{true}, \texttt{parameters} : \cdots, \texttt{exact_ground_energy} : E_{\text{exact,filtered}}, \texttt{trials} : [\cdots]\}.$$

For each trial, the summary contract includes

- `energy`,
- `exact_energy`,
- `delta_e`,
- `delta_E_abs`,
- `num_parameters`,
- `nfev`,

- `runtime_s`,
- ADAPT depth and stop-reason fields when the method kind is ADAPT.

The PDF contract is not only a raw command dump. It contains

1. a parameter manifest,
2. an executive summary,
3. a scoreboard table,
4. per-trial trajectory pages,
5. overlay appendix pages,
6. the executed command page.

So the cross-check suite is mathematically a multi-trial map

$$\tau \mapsto \left(E_\tau, \Delta E_\tau, \{\mathcal{O}_\tau(t_n)\}_{n=0}^{N_t-1}\right),$$

with a fixed exact target and fixed time grid.

Implemented surfaces:

- `pipelines/exact_bench/cross_check_suite.py`

14.6 HH seed-surface preset and proxy sidecars

For the current opt-in HH seed benchmark preset, the suite runs the four-point set

$$(L, g) \in \{2, 3\} \times \{0.8, 1.2\},$$

with shared HH choices

$$t = 1, \qquad U = 4, \qquad \omega_0 = 1, \qquad n_{\mathrm{ph,max}} = 1,$$

and the repository HH boundary/ordering convention.

For a warm-only baseline trial τ_0 and a refined seed trial τ, define the absolute-energy improvement

$$I(\tau \mid \tau_0) = |\Delta E_{\tau_0}| - |\Delta E_\tau|.$$

Let the additive preparation-cost deltas be

$$\Delta c_\tau = c_\tau - c_{\tau_0}, \qquad \Delta d_\tau = d_\tau - d_{\tau_0}, \qquad \Delta s_\tau = s_\tau - s_{\tau_0},$$

where (c_τ, d_τ, s_τ) denote the explicit `cx_proxy`, `depth_proxy`, and `sq_proxy` values persisted through the proxy sidecars.

Then the comparison views are

$$\eta_\tau^{(\mathrm{cx})} = \frac{I(\tau \mid \tau_0)}{\max(\Delta c_\tau, 1)}, \qquad \eta_\tau^{(\mathrm{depth})} = \frac{I(\tau \mid \tau_0)}{\max(\Delta d_\tau, 1)}, \qquad \eta_\tau^{(\mathrm{sq})} = \frac{I(\tau \mid \tau_0)}{\max(\Delta s_\tau, 1)}.$$

The decision rule uses $\eta_\tau^{(\mathrm{cx})}$ as the primary ranking metric, with the depth- and single-qubit-normalized views retained as secondaries. The proxy formulas themselves remain sourced from `docs/reports/qiskit_circuit_report.py` through `compute_sweep_proxy_cost(...)`, `compute_time_dynamics_proxy_cost(...)`, and the additive `ProxyMetricRow` sidecar schema.

15. Noise Validation, Symmetry-Mitigation, and Legacy-Parity Contracts

15.1 Filtered versus full exact targets

Code anchor: `pipelines/exact_bench/hh_noise_hardware_validation.py`

The noise-validation path stores two distinct exact energies:

$$E_{\text{exact,filtered}} = \min_{\psi \in \mathcal{S}_{\text{sector}}} \langle \psi | \hat{H} | \psi \rangle, \qquad E_{\text{exact,full}} = \min \operatorname{spec}(H_{\text{mat}}).$$

The filtered energy is the primary VQE comparison target in HH validation, while the full-Hilbert minimum is retained separately as provenance.

15.2 Noisy-minus-ideal observable deltas

Code anchor: `pipelines/exact_bench/hh_noise_hardware_validation.py`

For each trajectory observable `\mathcal O(t)` on the validation grid, the payload records a noisy value, an ideal value, and a difference field

$$\Delta \mathcal{O}(t) = \mathcal{O}_{\text{noisy}}(t) - \mathcal{O}_{\text{ideal}}(t).$$

In particular the validation runner explicitly stores

$$\Delta E_{\text{static}}(t) = E^{\text{noisy}}_{\text{static,trotter}}(t) - E^{\text{ideal}}_{\text{static,trotter}}(t),$$

$$\Delta D(t) = D^{\text{noisy}}_{\text{trotter}}(t) - D^{\text{ideal}}_{\text{trotter}}(t),$$

with companion stderr fields carried in the payload and plotted as uncertainty bands in the PDF.

At the VQE level the same noisy-minus-ideal logic is retained:

$$\Delta E_{\text{noise}} = E_{\text{noisy}} - E_{\text{ideal reference}}.$$

15.3 Legacy parity gate

Code anchor: `pipelines/exact_bench/hh_noise_hardware_validation.py`

The legacy parity helper compares the new ideal trajectory against a locked legacy reference trajectory.

For an observable `\mathcal O` on grid points `t_n`, the code computes

$$\delta^{\max}_{\mathcal{O}} = \max_n \left| \mathcal{O}_{\text{new,ideal}}(t_n) - \mathcal{O}_{\text{legacy}}(t_n) \right|,$$

$$\delta^{\text{mean}}_{\mathcal{O}} = \frac{1}{N_t} \sum_n \left| \mathcal{O}_{\text{new,ideal}}(t_n) - \mathcal{O}_{\text{legacy}}(t_n) \right|,$$

$$\delta^{\text{final}}_{\mathcal{O}} = \left| \mathcal{O}_{\text{new,ideal}}(t_{N_t-1}) - \mathcal{O}_{\text{legacy}}(t_{N_t-1}) \right|.$$

The pass condition is conjunctive:

$$\texttt{passed}_{\mathcal{O}} \iff (\texttt{time_grid_match} = \text{true}) \wedge (\delta^{\max}_{\mathcal{O}} \leq \texttt{tolerance}).$$

The global verdict is

$$\texttt{passed_all} \iff \texttt{time_grid_match} \wedge \bigwedge_{\mathcal{O} \in \mathcal{O}_{\text{cmp}}} \texttt{passed}_{\mathcal{O}}.$$

This is stricter than a pointwise visual comparison: an unequal time grid already fails the parity contract.

15.4 Same-anchor paired comparison semantics

Code anchor: `pipelines/exact_bench/hh_noise_hardware_validation.py`

The paired-anchor validation path evaluates one common workload / observable pair across several execution modes while retaining shared-anchor evidence.

Abstractly it evaluates a fixed tuple

$$(\hat{U}, \hat{O}, \text{lock family})$$

across several mode labels `m`, producing rows

$$r_m = \left(\text{mode}_m,\ \text{executor}_m,\ \text{noise kind}_m,\ \mu_m,\ \sigma_m,\ \text{layout/provenance hashes}_m\right).$$

The comparability payload then records whether these rows share anchor evidence such as

- backend identity,
- snapshot hash,
- layout hash,
- used physical qubits,
- used physical edges,
- circuit-structure hash,
- runtime-execution-bundle relationships.

So the same-anchor block is mathematically a controlled multi-mode comparison of one fixed workload rather than a loose comparison of different circuits.

15.5 Implemented symmetry-mitigation mode surface

Code anchor: `pipelines/exact_bench/hh_noise_hardware_validation.py`

The validation runner exposes the symmetry-mitigation mode names

$$\texttt{symmetry_mitigation_mode} \in \{\texttt{off}, \texttt{verify_only}, \texttt{postselect_diag_v1}, \texttt{projector_renorm_v1}\}.$$

These modes are normalized into the noisy-oracle configuration and, separately, into the ideal-reference symmetry-mitigation configuration.

This manuscript therefore states the implemented contract conservatively:

1. the mode names are live runtime inputs,
2. the chosen mode is retained in the validation provenance,
3. the PDF and JSON surfaces report the mitigation configuration,
4. the main-body mathematics here does not claim more internal estimator algebra than the code surfaces explicitly expose in this audit.

15.6 Manifest and provenance hashes

Code anchor: `pipelines/exact_bench/hh_noise_hardware_validation.py`

The validation payload tracks provenance through explicit hash and lock fields rather than only through human-readable text.

The recorded identifiers include

- `resolved_noise_spec_hash`,
- `snapshot_hash`,
- `layout_hash`,
- `transpile_hash`,
- `circuit_structure_hash`,
- `noise_artifact_hash`,
- `layout_anchor_source`,
- `runtime_execution_bundle`.

So the validation artifact is mathematically a tuple

$$\mathcal{V} = (\text{settings}, \text{filtered/full exact targets}, \text{noisy/ideal observables}, \text{legacy parity verdict}, \text{paired-anchor evidence}, \text{provenance hashe}$$

not just a trajectory dump.

Implemented surfaces:

- `pipelines/exact_bench/hh_noise_hardware_validation.py`

16. Final Primitive-Closed Summary

The full repository-aligned substitution chain is now linear and closed:

1. choose the fermion ordering `p(i,\sigma)`,
2. write the JW ladder primitives `\hat c_p^{\dagger}`, `\hat c_p`,
3. reduce `\hat n_p` to `(I-Z_p)/2`,
4. substitute those primitives into `\hat H_t`, `\hat H_U`, `\hat H_v`,
5. write `\hat b_i`, `\hat b_i^{\dagger}`, `\hat n_{b,i}`, `\hat x_i`,
6. substitute them into `\hat H_{\mathrm{ph}}` and `\hat H_g`,
7. reduce the drive term to explicit identity-plus-Z coefficients,
8. propagate those operators into the statevector primitives,
9. build the current hardcoded HH ansatz families from those same operators,
10. build PAOP and staged continuation objects from those same explicit primitives,
11. define the reported propagator, reference branch, filtered exact manifold, and branch-resolved observables on top of those same operators,
12. serialize the resulting continuation and state handoff with the implemented payload contract,
13. benchmark multiple conventional and ADAPT trial families against the same sector-filtered exact target on a common propagation grid,
14. validate noisy, ideal, and legacy trajectories with explicit parity gates and provenance hashes.

This is exactly the manuscript shape the current repository supports: linear, substitution-first, implementation-backed, and extended through the current propagation, benchmark, and validation contracts without promoting speculative future design claims into the main body.

Appendix A. Spec-only or not-located surfaces kept out of the main body

This appendix records material that appears in `MATH/IMPLEMENT_SOON.md` or `MATH/IMPLEMENT_NEXT.md` but is **not** promoted into the main manuscript as an implemented claim unless a direct code anchor was located in the scoped audit above.

A.1 Broader continuation-theory items

The two implementation-spec documents describe a broader continuation architecture, including items such as

- larger live selective macro-splitting frameworks,
- richer motif transfer and multi-source tiling,
- broader active symmetry-mitigation architecture,
- extended replay / rescue / continuation scoring theory.

Status in this manuscript:

- these ideas are **not** used as main-body source material;
- only runtime surfaces actually located in code, such as exposed mode names and handoff payloads, are promoted into present-tense chapters above.

A.2 Amplitude-comparison compare path

The drive-amplitude comparison compare-pipeline path is documented in `pipelines/run_guide.md`, including PDF and metrics-artifact descriptions.

Status in this audit:

- **not located in current scoped code search** across the inspected `pipelines/hardcoded/`, `pipelines/exact_bench/`, and `src/quantum/` surfaces;
- therefore it remains appendix-only here and is not stated as a present-tense implemented contract in the main manuscript.

A.3 Appendix rule

The operational rule for this manuscript is therefore:

1. main body = code-backed present-tense mathematics and payload/report contracts,
2. appendix = spec-only, design-note, or not-located material that should not be mistaken for implemented behavior in this checkout.